

基于几何光学投影的导弹视场遮蔽干扰弹投放策略优化

摘 要

在新时代强军目标指引下，无人机烟幕干扰技术作为电子对抗的关键环节，通过精准投放与空域防护，对提升我军作战优势和维护国家安全具有重要战略价值。

针对问题一：本文基于空间代数和动力学公式，建立了相关曲线的参数方程和点到直线距离表达式，进而得到约束条件；然后通过基于正多边形近似的仿真模拟方法求得满足有效遮蔽的时刻若干个时刻；最后利用微元法对这些时刻积零为整，进而得烟幕干扰弹对 $M1$ 的有效遮蔽时长为 1.3387s 。

针对问题二：本文把问题一中无人机的飞行方向等参数当成未知量。求解步骤和问题一几乎相同，首先建立相关曲线的参数方程和点到直线距离表达式；接着根据几何关系，构建所有参数满足的约束条件；本问可以建立最优化模型 $GA-2$ ，采取遗传算法（ GA ）对该模型进行求解并得到：当无人机 $FY1$ 飞行方向为与 x 轴正向夹角为 0.13rad 、飞行速度为 $96.7\text{m}\cdot\text{s}^{-1}$ 、烟雾干扰弹投放点为 $(17800, 0, 2800)$ 、烟幕干扰弹起爆点为 $(17882.29, 10.6, 1796.41)$ 时，最长遮蔽时间架为 4.53s 。

针对问题三：本文在问题二的基础上烟幕干扰弹的投放数量增加到了3枚。参考前两问的基本框架，以建立相关参数方程和点到直线距离关于参数的表达式并根据几何关系得出约束条件，从而建立最优化模型 $GA-3$ 。此处仍采用遗传算法（ GA ）对该模型进行求解，得到了最大遮蔽时长为 6.74s 下的烟幕干扰弹的投放策略，并把结果安全地保存到文件 `result1.xlsx` 中。

针对问题四：本文在问题二的基础上把无人机数量扩充到了3架。沿用前三问的基本思路，可以构造出相关的参数表达式，并得出相关约束条件，最终建立最优化模型 $GA-4$ 。使用遗传算法^[1]进行模型求解，得到最大遮蔽时长为 8.32s 下的烟幕干扰弹投放策略，并把结果安全地保存到文件 `result2.xlsx` 中。

针对问题五：本文在前四问的基础上把导弹数量增加到了3枚，无人机的数量扩充到了3架，每架无人机可投放的烟幕干扰弹数量增加到了至多3枚。综合考虑前四问的基本思路和基本框架，可以通过构建参数表达式以得到约束条件，最终通过遗传算法（ GA ），建立最优化模型 $GA-5$ ，找到最大遮蔽时间为 13.85s 下的近似全局最优解，并把烟幕干扰弹投放策略安全地保存到文件 `result3.xlsx` 中。

文章末尾，对本文的模型进行评价，针对第五问中无人机投放的烟幕干扰弹对导弹遮蔽程度过低这一现象，本文提出了改进方案以及修改方案。

关键词： 视场投影 遗传算法 微元法 最优化模型

一、问题重述

问题一考察单无人机执行干扰任务的情况。在仅有 FY1 无人机参与的情况下，假设该无人机以 120 m/s 的速度向假目标飞行，在任务受领后 1.5 秒投放烟幕干扰弹，并在延迟 3.6 秒后起爆。任务要求计算该烟幕弹对导弹 M1 的有效干扰时长。

问题二要求在 FY1 执行烟幕干扰的基础上，进一步协同优化无人机的最佳飞行方向和速度，并选择最佳的干扰弹投放点与起爆点。使得对导弹 M1 的干扰效果最优，从而最大化干扰弹的有效遮挡时间。

问题三涉及 FY1 单机执行多次投放的情境，其中无人机需连续投放三枚烟幕弹对导弹 M1 进行干扰，要求设计合理的投放时序和空间布局，使烟幕云团能够持续有效挡住目标，并确保干扰弹的遮蔽效果最优化。

问题四中三架无人机（FY1、FY2、FY3）协同干扰导弹 M1 且每架无人机投放一枚烟幕干扰弹，目标是通过优化各无人机的投放策略，延长烟幕的有效遮蔽时间，需要考虑无人机的协调性、投放位置的分布以及烟幕干扰弹的相互作用。

问题五要求五架无人机同时投放。每架最多投放三枚烟幕干扰弹，对三枚来袭导弹 M1、M2、M3 进行干扰，复杂程度和自由度均最高，要重点解决多目标分配、弹量约束下的投弹策略与起爆控制问题，以实现整体遮蔽效能最优。

二、模型假设

1. 假设导弹沿着直线飞行，忽略空气阻力等外力影响。
2. 假设烟幕干扰弹投射时，初速度矢量与投射该弹的无人机飞行速度相等。
3. 假烟幕云团形成圆锥形遮蔽区，遮挡目标视线。
4. 假设有效遮蔽为导弹与真目标上任意一点连线均与球状烟幕云团有公共点。
5. 假设环境因素对烟雾扩展与导弹飞行均无明显影响。
6. 假设导弹视野盲区为导弹与烟雾表面连线无法延伸至目标的区域。
7. 假设无人机在每次投放烟幕干扰弹时，需重新接收任务并调整飞行方向和速度。
8. 假设题目所给空间直角坐标系单位长度的距离是 1 米。

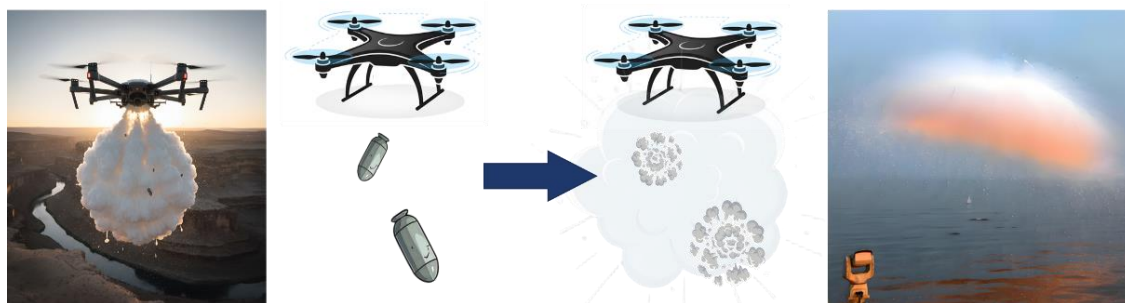


图 1 导弹发射烟幕团实际应用图示

三、问题分析

基于几何光学中“光沿直线传播”的原理，点光源照射球体时会在其后方投射形成阴影区域。针对本题可将导弹视作点光源，烟幕云团视作半径 10 米的实心球体，此时可以将烟幕云团对导弹的有效遮蔽区域边界看成一个动态圆锥的一部分。该圆锥的顶点位于实时导弹位置，圆锥侧面与烟幕云团表面相切形成有效遮蔽区域边界。^[2]为确保导弹无法看到真目标，圆柱体真目标需完全位于该有效遮蔽区域内。此时有效遮蔽时长即为圆柱体真目标完全处于有效遮蔽区域内的时间总和^[3]。

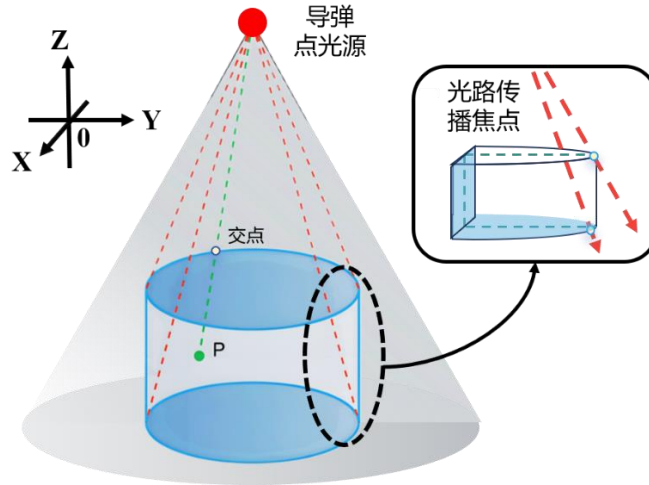


图 2 圆柱体在圆锥遮蔽区域内几何证明图

可以得到一个结论：真目标圆柱体的上下底面圆周均处于圆锥遮蔽区域内时，即可确保整个圆柱被有效遮蔽。此时只需验证该圆柱上下底面完全位于圆锥阴影内即可。

3.1 问题一 烟幕干扰弹的有效遮蔽时长评估的分析

为了求解有效遮蔽时长，可通过已知条件建立导弹 M_1 、烟幕干扰弹、烟幕云团球心的轨迹参数方程。其中，球心的方程起爆前是干扰弹的轨迹方程。接着计算球状烟幕云球心到真目标上下底面圆周上各点与导弹连线的距离。

通过限制此距离不大于球状烟幕的半径，可确定真目标处于有效遮蔽区域内的所有时刻。最后运用微分法^[4]将遮蔽时刻划分为最小时间微元，并将所有有效遮蔽的时刻求和，最终得到烟幕干扰弹对真实目标的有效遮蔽时长。

3.2 问题二 优化飞行参数以最大化遮蔽时间的分析

本问要求在遮蔽时间尽可能长的前提下，求解无人机 FY1 的飞行速度、飞行方向、烟幕干扰弹投放点、烟幕干扰弹起爆点等优化参数。。

在问题一的基础上，我们已经对上述参数进行设定，因此在本问中，我们仍沿用并求解相关轨迹参数方程。

在此框架下，球状烟幕球心到真目标上下底面圆周上各点与导弹所在直线的距离，就可以表示为已经发现导弹的时间、无人机的飞行速度、飞行方向与 x 轴正半轴夹角、烟幕干扰弹的投射时间、起爆时间的一个函数。可以利用遗传算法合理取值，寻找到能够使有效遮蔽时间达到最大的最优参数组合。

3.3 问题三 多枚干扰弹投放烟雾综合方案设计的分析

问题三在问题二基础上，将无人机 $FY1$ 投放烟幕干扰弹数量从1枚变成了3枚。同样采用先求出导弹 M_1 、烟幕干扰弹的轨迹参数方程，与3枚烟幕干扰弹起爆后的云团球心的轨迹参数方程及与导弹所在直线距离。

这样就可以通过限制3个球状烟幕球心到真目标上下底面圆周上的点与导弹所在直线距离不大于球状烟幕半径来得到3个不等式。根据题意，我们只需保证存在不等式成立，再根据遗传算法求解，即可得出答案。

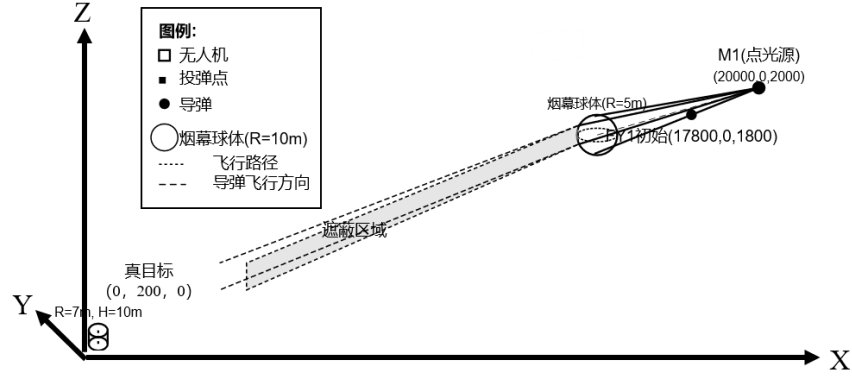


图3 单无人机几何光学导弹烟幕遮蔽示意图

3.4 问题四 三架无人机协同释放干扰情景优化

本问在问题二的基础上，增加无人机数量为3架，更改烟幕干扰弹为来自不同的无人机。对于该问题先求出导弹 M_1 和3枚烟幕干扰弹的轨迹参数方程以及弹起爆后球心的轨迹参数方程和与导弹所在直线距离。

这样就可以通过限制3个球状烟幕球心到真目标上下底面圆周上的点与导弹所在直线距离不大于球状烟幕半径^[5]，以得到3个不等式。根据不等式成立的限制条件，用遗传算法求解。

3.5 问题五 多架无人机协同干扰多枚导弹模型

本问在前四问的基础上，把无人机的数量扩展到5架，每架无人机投放幕干扰弹数不超过3枚，导弹数量增加到三枚，以此来制定烟幕干扰弹投放策略。我们可以先求出导弹 M_i 的轨迹参数方程，再求出无人机 FY_j 投放的第 k 枚烟幕干扰弹球心的轨迹参数方程，这样就能限制烟幕云团球心到直线距离不大于半径，从而建立最优化模型，采用遗传算法求解即可。

四、符号说明

符号	定义	单位
$M_i(x_{M_i}, y_{M_i}, z_{M_i})$	导弹 M_i 坐标	\
$d_{(i,j,k)}$	球心到直线距离	米
H	圆柱高度	米
$g_p(X)$	目标函数	秒
R	球状烟幕半径	米
R_0	圆柱底面圆周半径	米
Δt	有效遮蔽时长	秒

注：符号说明中未提到的符号，在本文中有具体解释

五、模型建立与求解

在本文研究中，我们针对烟幕干扰弹的投放策略建立了两类模型：第一问采用**仿真模拟**，通过**穷举**计算不同方案下效果，以获得最大时长；第二至第五问则采用**最优化模型**，利用**遗传算法**在较为复杂的多无人机、多导弹情形下进行策略优化，以延长真目标的有效遮盖时间。

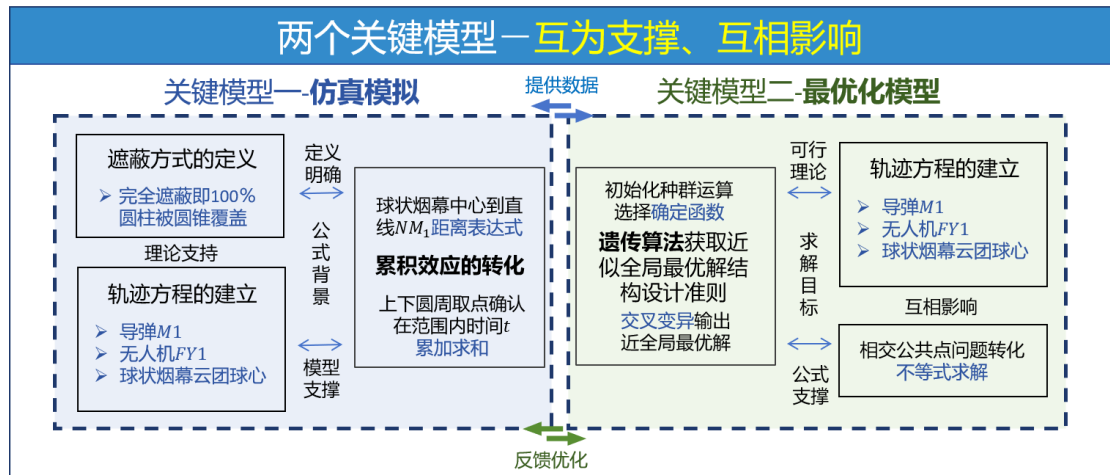


图4 本论文两个关键模型分析图

为统一表述，本文将遮挡方式定义为烟幕云团在导弹与目标之间形成的空间遮挡；将有效遮蔽定义为导弹飞行过程中与真目标连线均穿过烟幕遮蔽区域，从而使真目标在该时段内无法被发现。

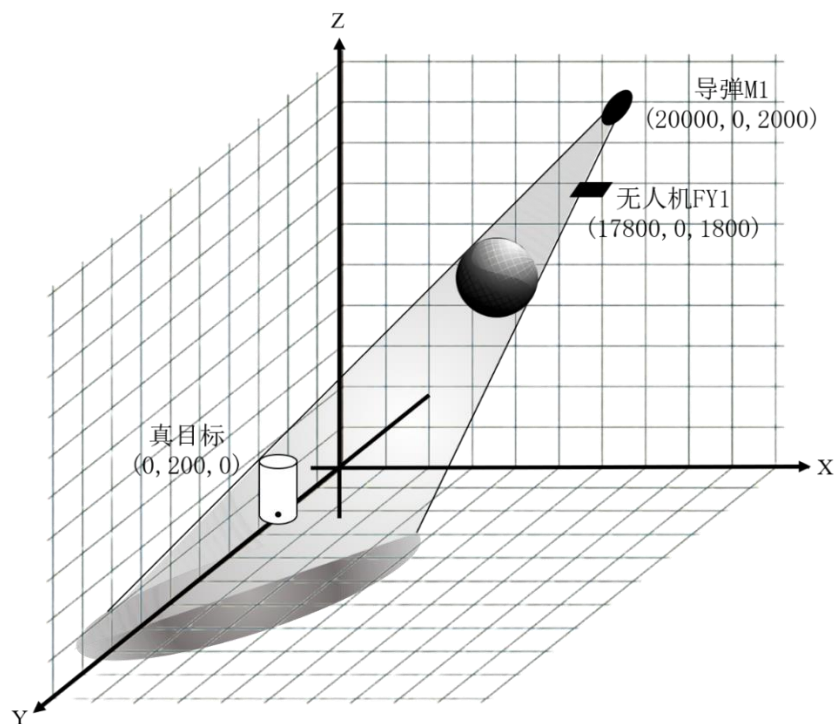


图 5 问题一二开始遮蔽示意图

5.1 问题一模型的建立与求解

5.1.1 模型建立仿真模拟

首先，根据题意建立导弹 $M1$ 的轨迹参数方程和无人机 $FY1$ 投放第 1 枚烟幕干扰弹以及干扰弹起爆后形成的球状烟幕云团球心的轨迹参数方程（以下简称球心轨迹方程）；再根据点到直线的距离公式求出球体中心点 $Q_{(1,1)}$ 到直线 NM_1 距离；根据微积分思想，得到有效遮蔽时长的表达式，进而进行仿真模拟。

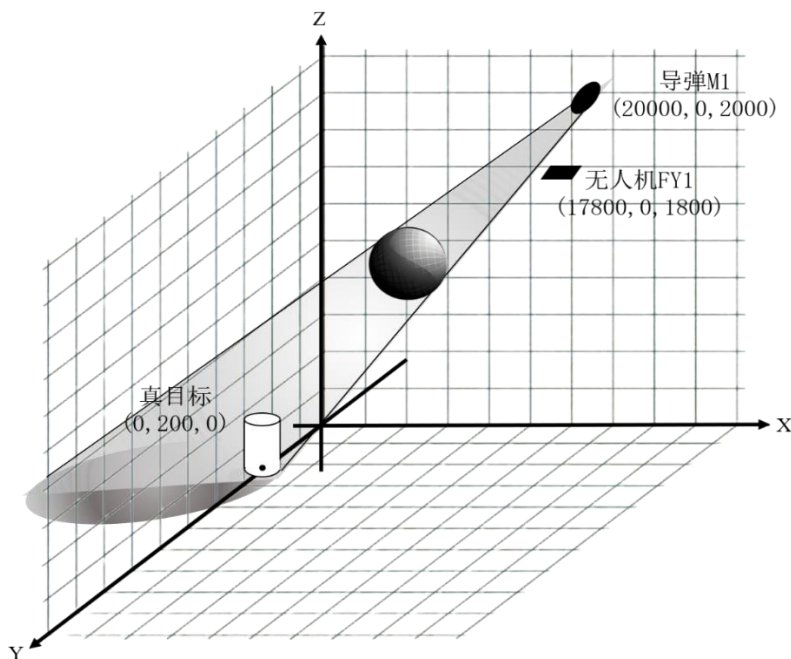


图 6 问题一二结束遮蔽示意图

5.1.1.1 导弹 M1、无人机 FY1 投放的干扰弹球心轨迹方程的建立

记 O 表示坐标原点；导弹 M_1 坐标为 $M_1(x_{M_1}, y_{M_1}, z_{M_1})$ ；速度为 v_D ；向量 $\overrightarrow{OM_1}$ 方向余弦为 $(\cos \alpha_{M_1}, \cos \beta_{M_1}, \cos \gamma_{M_1})$ ，则：

$$\cos \alpha_{M_1} = \frac{x_{M_1}}{\sqrt{x_{M_1}^2 + y_{M_1}^2 + z_{M_1}^2}}$$

$$\cos \beta_{M_1} = \frac{y_{M_1}}{\sqrt{x_{M_1}^2 + y_{M_1}^2 + z_{M_1}^2}}$$

$$\cos \gamma_{M_1} = \frac{z_{M_1}}{\sqrt{x_{M_1}^2 + y_{M_1}^2 + z_{M_1}^2}}$$

设 t 为已经发现导弹的时间，可以求出导弹 M_1 的轨迹参数方程：

$$\begin{cases} x'_{M_1}(t) = x_{M_1} - v_D t \cos \alpha_{M_1} \\ y'_{M_1}(t) = y_{M_1} - v_D t \cos \beta_{M_1} \\ z'_{M_1}(t) = z_{M_1} - v_D t \cos \gamma_{M_1} \end{cases}$$

记无人机 $FY1$ 接到投放第1枚干扰弹的命令时的坐标为 $F_{(1,1)}(x_{F_{(1,1)}}, y_{F_{(1,1)}}, z_{F_{(1,1)}})$ ；投放第1枚烟幕干扰弹的时间为 $t_0^{(1,1)}$ ；起爆时间为 $t_1^{(1,1)}$ ，则起爆间隔：

$$\Delta t_{(1,1)} = t_1^{(1,1)} - t_0^{(1,1)}$$

设无人机 $FY1$ 到投第1枚烟幕干扰弹的命令时的飞行速度为 $v_{F_{(1,1)}}$ ，飞行方向与 x 轴正方向夹角为 $\langle \overrightarrow{v_{F_{(1,1)}}}, \vec{i} \rangle = \theta_{(1,1)}$ ，从而得球心轨迹方程在不同定义域时如下：

当 $t \in [0, t_0^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} \end{cases}$$

当 $t \in (t_0^{(1,1)}, t_1^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} - \int_0^{t-t_0^{(1,1)}} g t dt \end{cases}$$

当 $t \in (t_1^{(1,1)}, t_2^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} - \int_0^{\Delta t_{(1,1)}} g t dt - v_L (t - t_1^{(1,1)}) \end{cases}$$

其中

$$t_2^{(1,1)} = \frac{z_{F_{(1,1)}} - \int_0^{\Delta t_{(1,1)}} g t dt}{v_L} + t_1^{(1,1)}$$

5.1.1.2 球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离表达式的建立

设无人机 $FY1$ 投放第1枚烟幕干扰弹起爆后形成球心为：

$$Q_{(1,1)}(x_{(1,1)}, y_{(1,1)}, z_{(1,1)})$$

设 R_0 为圆柱半径， H 为圆柱高度， (x_z, y_z, z_z) 为圆柱底面圆周坐标，则真目标圆柱上下圆周上的点 $N(x_N, y_N, z_N)$ 满足：

$$\begin{cases} (x_N - x_z)^2 + (y_N - y_z)^2 \leq R_0^2 \\ z_N = 0, H \end{cases}$$

则点 N 与点 $Q_{(1,1)}$ 连线构成向量 $\overline{NQ_{(1,1)}}$ ，且

$$\overline{NQ_{(1,1)}} = (x_{(1,1)} - x_N, y_{(1,1)} - y_N, z_{(1,1)} - z_N)$$

同时点 N 与点 M_1 连线构成向量 $\overline{NM_1}$ ，且

$$\overline{NM_1} = (x_{M_1} - x_N, y_{M_1} - y_N, z_{M_1} - z_N)$$

从而得到球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离

$$d_{(1,1)} = \sqrt{|\overline{NQ_{(1,1)}}|^2 - \left(\frac{\overline{NQ_{(1,1)}} \cdot \overline{NM_1}}{|\overline{NM_1}|} \right)^2}$$

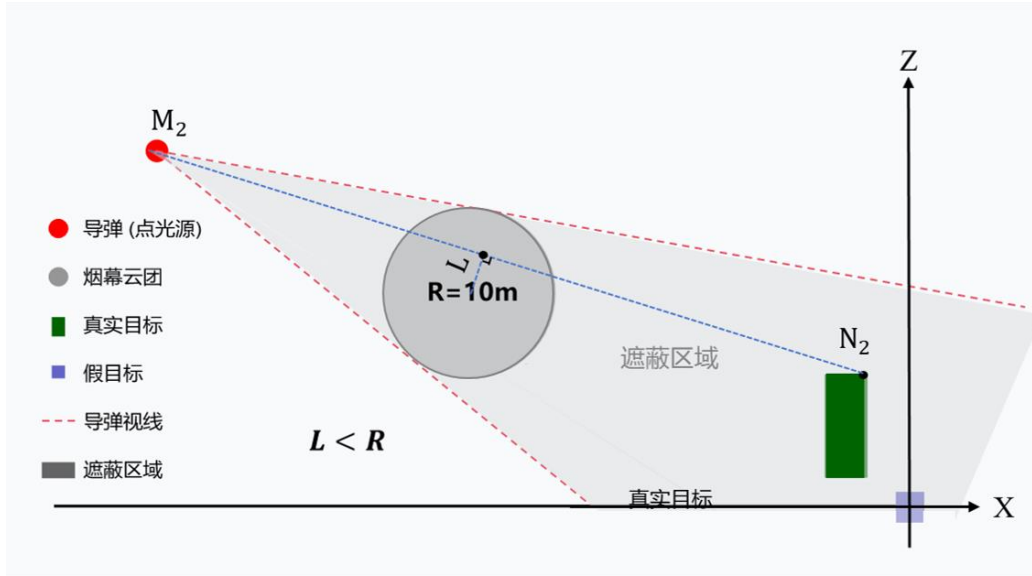


图 7 烟幕干扰弹有效遮蔽真目标原理侧视图

5.1.1.3 遍历模型 BL 的建立

圆柱上下园周上的点和导弹所连线段与球状烟幕有公共点，只要保证球体中心点 $Q_{(1,1)}$ 到直线 NM_i 距离不大于球半径 $R = 10m$ ，也即：

$$d_{(1,1,1)} \leq R$$

($d_{(1,1,1)} \leq R$ 示意图如图 7 所示；反之 $d_{(1,1,1)} > R$ 示意图如图 8 所示)

设烟幕干扰弹的理论最大有效遮蔽时间为 $\Delta t'$ ，把已经发现导弹的时间 t 从 $t = t_1^{(1,1)}$ 开始遍历，到 $t = t_1^{(1,1)} + \Delta t'$ 截止，最终得到满足 $d_{(1,1,1)} \leq R$ 的若干个时间段 $\Delta t_l (l \in N^+)$ ，则烟幕干扰弹对 M_1 的有效遮蔽时长 Δt 为

$$\Delta t = \sum_l \Delta t_l$$

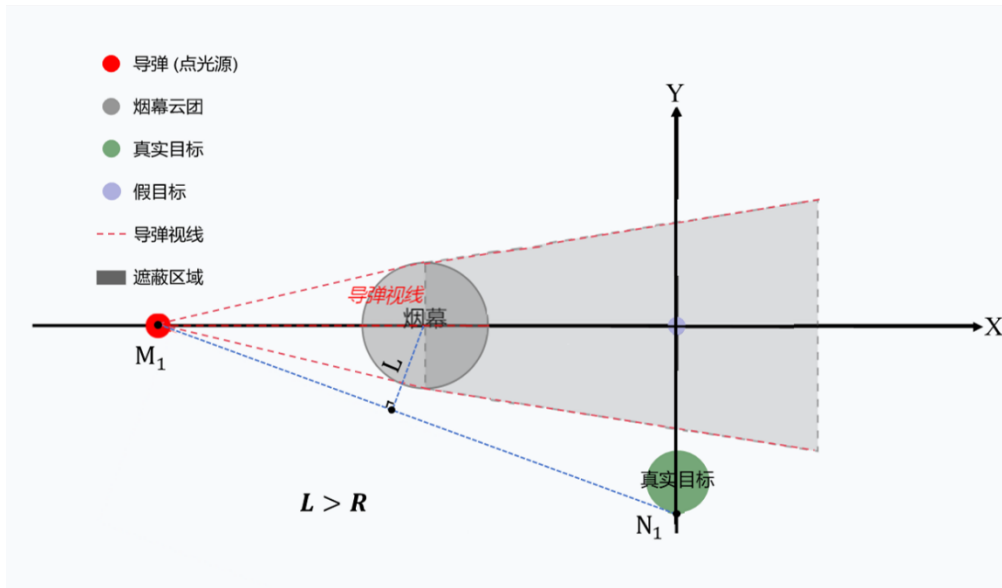


图 8 烟幕干扰弹非有效遮蔽真目标原理俯视图

这样就能够建立模型并使用仿真模拟进行求解。通过对时间 t 建立方程，得到烟幕干扰弹对 M_1 的最大有效遮蔽时长。

5.1.2 使用仿真模拟方法求解

由于圆周是一个连续的图形，圆周上每一个点都对应唯一坐标，则连续的圆周上对应着无数个点 and 无数个坐标。

由于计算机无法处理无限多组数据，因此我们对圆周上的点进行**均匀角度采样**（**Uniform Angular Sampling**），用**正多边形近似**（**Regular Polygon Approximation**）来逼近整个圆周，把连续性累计问题转化为离散化瞬态问题。因此不妨在上下圆周上各取10000个点，设这些点的坐标为 $N_{(m,z_{Nm})}(x_{Nm}, y_{Nm}, z_{Nm})$ ， $m=1,2,3,...,10000$ ，有

$$\begin{cases} x_{Nm} = R_0 \cos \frac{2\pi}{m} + x_z \\ y_{Nm} = R_0 \sin \frac{2\pi}{m} + y_z \\ z_{Nm} = 0, 1 \end{cases}$$

只要这 20000 个点全部在有效遮蔽范围内，就能确定该时刻圆柱体真目标全部在有效遮蔽范围内。

把时间 t 在烟幕干扰弹从起爆到遮蔽失效这段时间内，即区间 $[t_1^{(1,1)}, t_1^{(1,1)} + \Delta t']$ 内，以 $dt = 0.0001$ 为时间步长进行遍历，得到 l 个有效遮蔽时间段 $\Delta t_l (l \in N^+)$ ，最终求和得到烟幕干扰弹对 $M1$ 的有效遮蔽时长 Δt 为：

$$\Delta t = \sum_l \Delta t_l = 1.3387s$$

5.2 问题二最优化模型的建立与求解

5.2.1 最优化模型 GA-2 的建立

首先，根据题意建立导弹 $M1$ 的轨迹参数方程和无人机 $FY1$ 投放第1枚烟幕干扰弹以及干扰弹起爆后形成的球状烟幕云团球心的轨迹参数方程；接着，根据点到直线的距离公式求出球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离；最后，根据题意建立最优化模型 $GA-2$ 。

5.2.1.1 导弹 $M1$ 、无人机 $FY1$ 投放第1枚烟幕弹及球心轨迹方程的建立

设 t 为已经发现导弹的时间，可以求出导弹 $M1$ 的轨迹参数方程：

$$\begin{cases} x'_{M_1}(t) = x_{M_1} - v_D t \cos \alpha_{M_1} \\ y'_{M_1}(t) = y_{M_1} - v_D t \cos \beta_{M_1} \\ z'_{M_1}(t) = z_{M_1} - v_D t \cos \gamma_{M_1} \end{cases}$$

无人机FY1投放第1枚烟幕干扰弹以及干扰弹起爆后形成的球状烟幕云团球心的轨迹参数方程为：

当 $t \in [0, t_0^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} - \int_0^{t-t_0^{(1,1)}} g t dt \end{cases}$$

当 $t \in (t_0^{(1,1)}, t_1^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} - \int_0^{t-t_0^{(1,1)}} g t dt \end{cases}$$

当 $t \in (t_1^{(1,1)}, t_2^{(1,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,1)}(t) = x_{F_{(1,1)}} + v_{F_{(1,1)}} \cos \theta_{(1,1)} t \\ y_{(1,1)}(t) = y_{F_{(1,1)}} + v_{F_{(1,1)}} \sin \theta_{(1,1)} t \\ z_{(1,1)}(t) = z_{F_{(1,1)}} - \int_0^{\Delta t_{(1,1)}} g t dt - v_L (t - t_1^{(1,1)}) \end{cases}$$

其中

$$t_2^{(1,1)} = \frac{z_{F_{(1,1)}} - \int_0^{\Delta t_{(1,1)}} g t dt}{v_L} + t_1^{(1,1)}$$

5.2.1.2 球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离表达式的建立

真目标圆柱上下圆周上的点 $N(x_N, y_N, z_N)$ 满足：

$$\begin{cases} x_N^2 + y_N^2 \leq R_0^2 \\ z_N = 0, H \end{cases}$$

点 N 与点 $Q_{(1,1)}$ 连线构成向量 $\overrightarrow{NQ_{(1,1)}}$ ，同时点 N 与点 M_1 连线构成向量 $\overrightarrow{NM_1}$

$$\overrightarrow{NQ_{(1,1)}} = (x_{(1,1)} - x_N, y_{(1,1)} - y_N, z_{(1,1)} - z_N)$$

$$\overrightarrow{NM_1} = (x_{M_1} - x_N, y_{M_1} - y_N, z_{M_1} - z_N)$$

从而得到球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离

$$d_{(1,1,1)} = \sqrt{\left| \overline{NQ_{(1,1)}} \right|^2 - \left(\frac{\overline{NQ_{(1,1)}} \cdot \overline{NM_1}}{\left| \overline{NM_1} \right|} \right)^2}$$

其中

$$d_{(1,1,1)} = f\left(t, v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)}\right)$$

5.2.1.3 最优化模型 GA-2 的建立

要保证圆柱上下圆周上的点和导弹所连线段与烟幕干扰弹起爆后形成的半径 $R=10m$ 球状云团有公共点，只需保证球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离不大于球半径 R ，也即：

$$d_{(1,1,1)} \leq R$$

从而

$$f\left(t, v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)}\right) \leq R$$

设满足 $d_{(1,1,1)} \leq R$ 的若干个时间段为 $\Delta t_l (l \in N^+)$ ，则烟幕干扰弹对 M_1 的有效遮蔽时长 Δt 为

$$\Delta t = \sum_l \Delta t_l$$

我们可以通过遗传算法对矩阵 $X = (v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)})$ 进行合理取值，这样，对于每一个矩阵 X ，都能得到与之一一对应的有效遮蔽时长 Δt ，也就是说我们能得到 Δt 关于矩阵 $X = (v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)})$ 的函数，也即：

$$\Delta t = g(X) = h\left(v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)}\right)$$

设 X_{lower} 为矩阵 X 中参数的下确界组成的矩阵， X_{upper} 为矩阵 X 中参数的上确界组成的矩阵，也即：

$$X_{lower} \leq X \leq X_{upper}$$

该矩阵满足（忽略单位）

$$\begin{cases} X_{lower} = [70, 0, 0, 0] \\ X_{upper} = [140, 2\pi, 20, 28] \end{cases}$$

因此我们可以建立最优化模型 GA-1，其中，决策变量和目标函数分别为：

$$X = (v_{F_{(1,1)}}, \theta_{(1,1)}, t_0^{(1,1)}, t_1^{(1,1)}); \text{ s.t. } X_{lower} \leq X \leq X_{upper};$$

$$\max \Delta t = g(X)$$

5.2.2 最优化模型 GA-2 的求解

为了克服暴力搜索算法（Brute-force Search）因组合溢出导致的困难，我们采取遗传算法（GA），通过自然选择机制，来实现近似全局最优解的高效搜索。

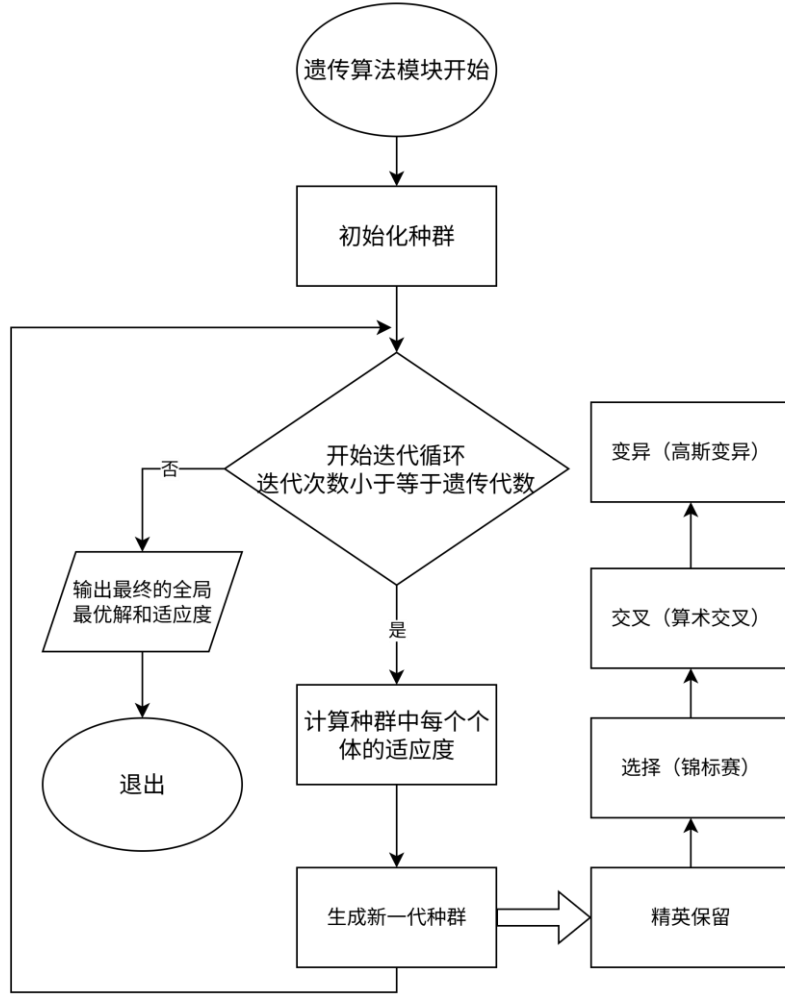


图 9 遗传算法流程图

Step1.确定实数编码策略

为了便于较大空间的遗传搜索、提高遗传算法的精度,我们采取实数编码,用有序实数组 $(v_{F_{(1,i)}}, \theta_{(1,i)}, t_0^{(1,1)}, t_1^{(1,1)})$ 作为染色体,各个实数取值与决策变量相同。

Step2.初始化种群

先定义种群数目为 600, 迭代次数为 250; 然后, 用均匀随机初始化算法 (Uniform Random Initialization), 求得初始种群。

Step3.确定适应度函数

在 GA-2 最优化模型中, 目标函数为有效遮蔽时间, 取目标函数作为适应度函数, 要求:

$$\max \Delta t = h(v_{F_{(1,i)}}, \theta_{(1,i)}, t_0^{(1,1)}, t_1^{(1,1)})$$

Step4.进行选择运算

运用锦标赛选择法(*Tournament Selection*), 随机挑选 3 个个体进行比赛; 同时通过精英保留法(*Elistism*), 选取适应度高的 2 个个体, 并自我复制进入下一代; 未被保留的个体淘汰。

Step5.进行线性算数交叉运算

由于染色体上基因实数组是有序的, 因此我们对父代个体基因进行线性算数交叉运算, 以确定子代个体。

运算步骤如下: 设父代个体中位于同一位置的两个基因对应的实数为 A, B , 线性交叉指数为 α (取 $\alpha = 0.8$), 则子代该位置基因对应实数为 $C = A\alpha + B(1 - \alpha)$ 。

Step6.进行高斯变异 (Gaussian Mutation) 运算

对所有染色体上基因对应的实数, 生成 (0,1) 之间的随机数, 若随机数不大于变异概率 0.16, 则该对基因进行高斯变异^[5] (*Gaussian Mutation*), 通过引入局部扰动增强算法的局部搜索能力, 以便于在最优解附近进行精细搜索。

高斯变异方式为: 给即将变异的基因值添加一个符合高斯分布 (正态分布) 的随机数。设原基因对应实数为 a_0 , 变异后该基因所对应实数为 a_1 , $Range$ 为决策变量取值的区间长度组成的矩阵, 放缩倍数 $\beta = 0.1$, 则正态分布标准差 (放缩因子) 为 $\sigma = \beta \cdot Range$, 从而得到高斯变异公式为:

$$a_1 = a_0 + N(0, \sigma)$$

Step7.终止条件判定与最优解输出

当预设达到最大迭代次数或种群收敛时, 跳出循环, 算法终止, 输出当前种群中适应度最高的个体最为全局近似最优解。结果如图表所示。

表 1 模型 GA-2 的最优解输出

变量名称	取值
无人机速度	96.7m/s
投放时间	0
起爆时间	0.86s
偏航角	0.13rad
最大遮蔽时长	4.53s
投放点坐标	(17800,0,2800)
起爆点坐标	(17882.29,10.6,1796.41)

5.3 问题三模型的建立与求解

5.3.1 最优化模型 GA-3 的建立

首先，根据题意建立导弹 $M1$ 的轨迹参数方程和无人机 $FY1$ 投放第 k 枚烟幕干扰弹以及干扰弹起爆后形成的球状烟幕云团球心的轨迹参数方程；接着，根据点到直线的距离公式求出球状烟幕中心点 $Q_{(1,k)}$ 到直线 NM_1 距离；最后，根据题意建立最优化模型 $GA-3$ 。

5.3.1.1 导弹 $M1$ 、无人机 $FY1$ 投第 k 枚干扰弹及球心轨迹参数方程的建立

记无人机 FYj ($j=1,2,3,4,5$) 投放第 k ($k=1,2,3\dots$) 枚烟幕干扰弹的时间为 $t_0^{(j,k)}$ ，起爆时间为 $t_1^{(j,k)}$ ，则起爆间隔 $\Delta t_{(j,k)} = t_1^{(j,k)} - t_0^{(j,k)}$

设 t 为已经发现导弹的时间，可以求出导弹 $M1$ 的轨迹参数方程：

$$\begin{cases} x'_{M_1}(t) = x_{M_1} - v_D t \cos \alpha_{M_1} \\ y'_{M_1}(t) = y_{M_1} - v_D t \cos \beta_{M_1} \\ z'_{M_1}(t) = z_{M_1} - v_D t \cos \gamma_{M_1} \end{cases}$$

记无人机 FYj 接到投放第 k 枚烟幕干扰弹的命令时的坐标为 $F_{(j,k)}(x_{F_{(j,k)}}, y_{F_{(j,k)}}, z_{F_{(j,k)}})$ ；根据题意 $F_{(j,1)}$ 和 $z_{F_{(j,k)}}$ 已知，且当 $k \geq 2$ 时，有

$$\begin{cases} x_{F_{(j,k)}} = x_{F_{(j,k-1)}} + v_{F_{(j,k)}} \cos \theta_{(j,k)} \\ y_{F_{(j,k)}} = y_{F_{(j,k-1)}} + v_{F_{(j,k)}} \sin \theta_{(j,k)} \end{cases}$$

记无人机 FYj 的，投放第 k 枚烟幕干扰弹时无人机 FYj 飞行方向与 x 轴正方向夹角为 $\langle \overrightarrow{v_{F_{(j,k)}}}, \vec{i} \rangle = \theta_{(j,k)}$ ，飞行速度为 $v_{F_{(j,k)}}$

无人机 $FY1$ 投放第 k 枚烟幕干扰弹以及球心不同定义域的轨迹方程为：

当 $t \in [0, t_0^{(1,k)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,k)}(t) = x_{F_{(1,k)}} + v_{F_{(1,k)}} \cos \theta_{(1,k)} t \\ y_{(1,k)}(t) = y_{F_{(1,k)}} + v_{F_{(1,k)}} \sin \theta_{(1,k)} t \\ z_{(1,k)}(t) = z_{F_{(1,k)}} \end{cases}$$

当 $t \in (t_0^{(1,k)}, t_1^{(1,k)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,k)}(t) = x_{F_{(1,k)}} + v_{F_{(1,k)}} \cos \theta_{(1,k)} t \\ y_{(1,k)}(t) = y_{F_{(1,k)}} + v_{F_{(1,k)}} \sin \theta_{(1,k)} t \\ z_{(1,k)}(t) = z_{F_{(1,k)}} - \int_0^{t-t_0^{(1,k)}} g t dt \end{cases}$$

当 $t \in (t_1^{(1,k)}, t_2^{(1,k)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(1,k)}(t) = x_{F_{(1,k)}} + v_{F_{(1,k)}} \cos \theta_{(1,k)} t \\ y_{(1,k)}(t) = y_{F_{(1,k)}} + v_{F_{(1,k)}} \sin \theta_{(1,k)} t \\ z_{(1,k)}(t) = z_{F_{(1,k)}} - \int_0^{t-t_1^{(1,k)}} g t dt - v_L (t - t_1^{(1,k)}) \end{cases}$$

其中

$$t_2^{(1,k)} = \frac{z_{F_{(1,k)}} - \int_0^{\Delta t_{(1,k)}} g t dt}{v_L} + t_1^{(1,k)}$$

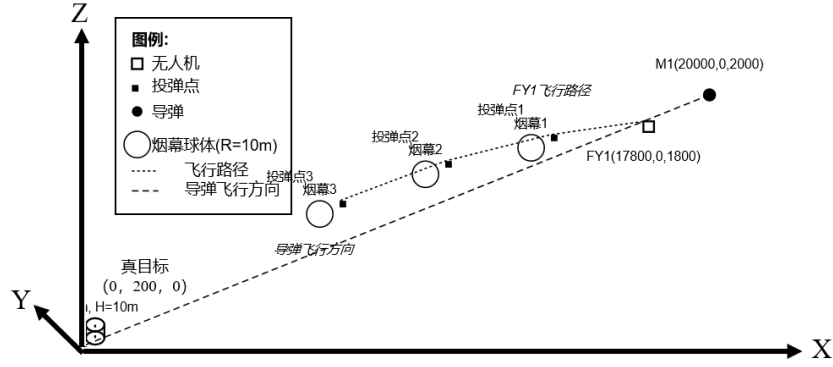


图 10 单无人机多投放弹几何光学遮蔽示意图

5.3.1.2 球状烟幕中心点 $Q_{(1,k)}$ 到直线 NM_1 距离表达式的建立

设无人机 FY1 投放第 k 枚烟幕干扰弹起爆后形成的球状烟幕云团球心为：

$$Q_{(1,k)}(x_{(1,k)}, y_{(1,k)}, z_{(1,k)})$$

真目标圆柱上下圆周上的点 $N(x_N, y_N, z_N)$ 满足：

$$\begin{cases} x_N^2 + y_N^2 \leq R_0^2 \\ z_N = 0, H \end{cases}$$

点 N 与点 $Q_{(1,k)}$ 连线构成向量 $\overline{NQ_{(1,k)}}$ ，且

$$\overline{NQ_{(1,k)}} = (x_{(1,k)} - x_N, y_{(1,k)} - y_N, z_{(1,k)} - z_N)$$

同时点 N 与点 M_1 连线构成向量 $\overline{NM_1}$ ，且

$$\overline{NM_1} = (x_{M_1} - x_N, y_{M_1} - y_N, z_{M_1} - z_N)$$

从而得到球状烟幕中心点 $Q_{(1,k)}$ 到直线 NM_1 距离

$$d_{(1,k)} = \sqrt{|\overline{NQ_{(1,k)}}|^2 - \left(\frac{\overline{NQ_{(1,k)}} \cdot \overline{NM_1}}{|\overline{NM_1}|} \right)^2}$$

其中

$$d_{(1,1,k)} = f\left(t, v_{F_{(1,k)}}, \theta_{(1,k)}, t_0^{(1,k)}, t_1^{(1,k)}\right)$$

5.3.1.3 最优化模型 GA-3 的建立

圆柱上下圆周上的点和导弹所连线段与烟幕干扰弹起爆后形成的半径 $R=10m$ 球状云团有公共点，只要保证球状烟幕中心点 $Q_{(1,1)}$ 到直线 NM_1 距离不大于球半径 R ，也即：

$$d_{(1,1,k)} \leq R$$

从而

$$f\left(t, v_{F_1}, \theta_1, t_0^{(1,k)}, t_1^{(1,k)}\right) \leq R$$

设满足 $d_{(1,1,k)} \leq R$ 的若干个时间段为 Δt_l ($l \in N^+$)，则烟幕干扰弹对 M_1 的有效遮蔽时长 Δt 为

$$\Delta t = \sum_l \Delta t_l$$

我们可以通过遗传算法对矩阵 $X = (v_{F_{(1,k)}}, \theta_{(1,k)}, t_0^{(1,k)}, t_1^{(1,k)})$ 进行合理取值，这样，对于每一个矩阵 X ，都能得到与之一一对应的有效遮蔽时长 Δt ，也就是说我们能得到 Δt 关于矩阵 $X = (v_{F_{(1,k)}}, \theta_{(1,k)}, t_0^{(1,k)}, t_1^{(1,k)})$ 的函数，也即：

$$\Delta t = g_1(X) = h_1\left(v_{F_{(1,k)}}, \theta_{(1,k)}, t_0^{(1,k)}, t_1^{(1,k)}\right)$$

设 X_{lower} 为矩阵 X 中参数的下确界组成的矩阵， X_{upper} 为矩阵 X 中参数的上确界组成的矩阵，也即：

$$X_{lower} \leq X \leq X_{upper}$$

并满足（忽略单位）

$$\begin{cases} X_{lower} = [70, 0, 0, 0, 70, 0, 1, 0, 70, 0, 2, 0] \\ X_{upper} = [140, 2\pi, 10, 18, 140, 2\pi, 25, 33, 140, 2\pi, 35, 43] \end{cases}$$

因此我们可以建立最优化模型 GA-2，其中：

$$\text{决策变量 } X = (v_{F_{(1,k)}}, \theta_{(1,k)}, t_0^{(1,k)}, t_1^{(1,k)}) ; \quad \text{s.t. } X_{lower} \leq X \leq X_{upper} ;$$

$$\text{目标函数 } \max_{\Delta t} \Delta t = g_2(X)$$

5.3.2 最优化模型 GA-3 的求解

第三问的求解沿用第二问建立的遗传框架，基础步骤与算法流程均保持不变。下面只列出第三问遗传算法（GA）与第二问中不同的关键参数和基本要点。

表 2 模型 $GA-3$ 的相关参数和实数编码对象

实数编码对象	目标函数 g_2 决策变量
种群数目	600
迭代次数	350
线性交叉指数	0.86
基因变异概率	0.17

通过以上参数对模型 $GA-3$ 进行求解，得到三枚烟雾弹的不同投放策略，最终结果得到最大有效遮蔽总时间为 6.74s，并将投放策略保存在文件 result1.xlsx 中。遗传算法进化过程如图 11 所示。

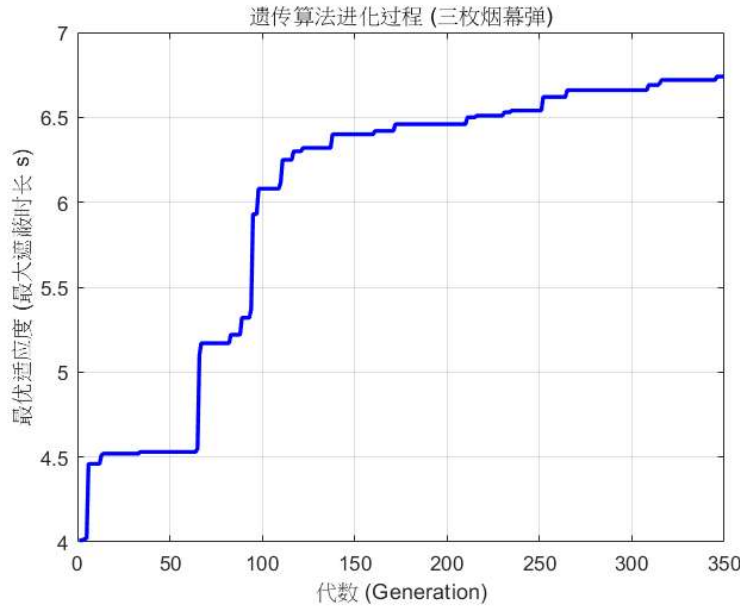


图 11 问题三遗传算法进化过程示意图

5.4 问题四模型的建立与求解

5.4.1 最优化模型 $GA-4$ 的建立

首先，根据题意建立导弹 $M1$ 的轨迹参数方程和无人机 FYj 投放第 1 枚烟幕干扰弹球心的轨迹参数方程；接着，根据点到直线的距离公式求出球状烟幕中心点 $Q_{(j,1)}$ 到直线 NM_1 距离；最后，根据题意建立最优化模型 $GA-3$

5.4.1.1 导弹 $M1$ 的、无人机 FYj 投放第 1 枚干扰弹球心参数方程的建立

设 t 为已经发现导弹的时间，可以求出导弹 $M1$ 的轨迹参数方程：

$$\begin{cases} x'_{M_1}(t) = x_{M_1} - v_D t \cos \alpha_{M_1} \\ y'_{M_1}(t) = y_{M_1} - v_D t \cos \beta_{M_1} \\ z'_{M_1}(t) = z_{M_1} - v_D t \cos \gamma_{M_1} \end{cases}$$

从而得到：

无人机 FY_j 投放第1枚烟幕干扰弹以及干扰弹起爆后形成的球状烟幕云团球心的轨迹参数方程为：

当 $t \in [0, t_0^{(j,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(j,1)}(t) = x_{F_{(j,1)}} + v_{F_{(j,1)}} \cos \theta_{(j,1)} t \\ y_{(j,1)}(t) = y_{F_{(j,1)}} + v_{F_{(j,1)}} \sin \theta_{(j,1)} t \\ z_{(j,1)}(t) = z_{F_{(j,1)}} \end{cases}$$

当 $t \in (t_0^{(j,1)}, t_1^{(j,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(j,1)}(t) = x_{F_{(j,1)}} + v_{F_{(j,1)}} \cos \theta_{(j,1)} t \\ y_{(j,1)}(t) = y_{F_{(j,1)}} + v_{F_{(j,1)}} \sin \theta_{(j,1)} t \\ z_{(j,1)}(t) = z_{F_{(j,1)}} - \int_0^{t-t_0^{(j,1)}} g t dt \end{cases}$$

当 $t \in (t_1^{(j,1)}, t_2^{(j,1)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(j,1)}(t) = x_{F_{(j,1)}} + v_{F_{(j,1)}} \cos \theta_{(j,1)} t \\ y_{(j,1)}(t) = y_{F_{(j,1)}} + v_{F_{(j,1)}} \sin \theta_{(j,1)} t \\ z_{(j,1)}(t) = z_{F_{(j,1)}} - \int_0^{t-t_0^{(j,1)}} g t dt - v_L (t - t_1^{(j,1)}) \end{cases}$$

其中

$$t_2^{(j,1)} = \frac{z_{F_{(j,1)}} - \int_0^{\Delta t_{(j,1)}} g t dt}{v_L} + t_1^{(j,1)}$$

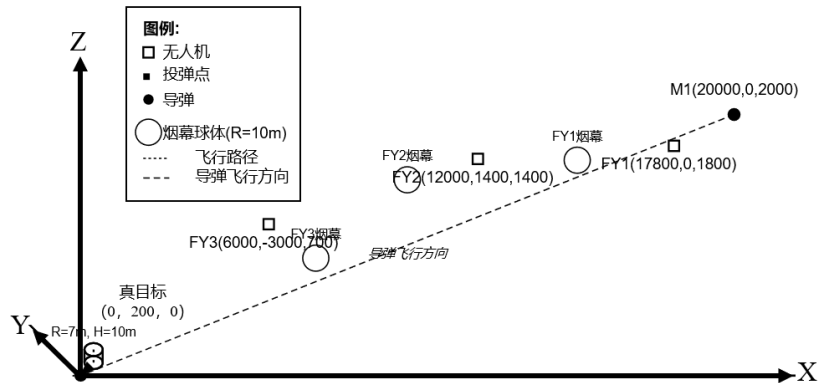


图 12 多无人机单投放弹几何光学遮蔽示意图

5.4.1.2 球状烟幕中心点 $Q_{(j,1)}$ 到直线 NM_1 距离表达式的建立

真目标圆柱上下圆周上的点 $N(x_N, y_N, z_N)$ 满足：

$$\begin{cases} x_N^2 + y_N^2 \leq R_0^2 \\ z_N = 0, H \end{cases}$$

设无人机 FYj 投放第1枚烟幕干扰弹起爆后形成的球状烟幕云团球心为：

$$Q_{(j,1)}(x_{(j,1)}, y_{(j,1)}, z_{(j,1)})$$

点 N 与点 $Q_{(j,1)}$ 连线构成向量 $\overrightarrow{NQ_{(j,1)}}$ ，且

$$\overrightarrow{NQ_{(j,1)}} = (x_{(j,1)} - x_N, y_{(j,1)} - y_N, z_{(j,1)} - z_N)$$

同时点 N 与点 M_1 连线构成向量 $\overrightarrow{NM_1}$ ，且

$$\overrightarrow{NM_1} = (x_{M_1} - x_N, y_{M_1} - y_N, z_{M_1} - z_N)$$

从而得到球状烟幕中心点 $Q_{(j,1)}$ 到直线 NM_1 距离

$$d_{(1,j,1)} = \sqrt{|\overrightarrow{NQ_{(j,1)}}|^2 - \left(\frac{\overrightarrow{NQ_{(j,1)}} \cdot \overrightarrow{NM_1}}{|\overrightarrow{NM_1}|} \right)^2}$$

其中

$$d_{(1,j,1)} = f(t, v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)})$$

5.4.1.3 最优化模型 GA-4 的建立

圆柱上下圆周上的点和导弹所连线段与烟幕干扰弹起爆后形成的半径 $R=10m$ 球状云团有公共点^[6]，只要保证球状烟幕中心点 $Q_{(j,1)}$ 到直线 NM_1 距离不大于球半径 R ，也即：

$$d_{(1,j,1)} \leq R$$

从而

$$f(t, v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)}) \leq R$$

设满足 $d_{(1,j,1)} \leq R$ 的若干个时间段为 $\Delta t_l (l \in N^+)$ ，则烟幕干扰弹对 M_1 的有效遮蔽时长 Δt 为

$$\Delta t = \sum_l \Delta t_l$$

我们可以通过遗传算法对矩阵 $X = (v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)})$ 进行合理取值，这样，对于每一个矩阵 X ，都能得到与之一一对应的有效遮蔽时长 Δt ，也就是说我们能得到 Δt 关于矩阵 $X = (v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)})$ 的函数，也即：

$$\Delta t = g(X) = h(v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)})$$

设 X_{lower} 为矩阵 X 中参数的下确界组成的矩阵， X_{upper} 为矩阵 X 中参数的上确界组成的矩阵，也即：

$$X_{lower} \leq X \leq X_{upper}$$

并满足（忽略单位，且只列出了重复）

$$\begin{cases} X_{lower} = [70, 0, 0, 0] \\ X_{upper} = [140, 2\pi, 15, 23] \end{cases}$$

因此我们可以建立最优化模型 $GA-4$ ，其中决策变量 $X = (v_{F(j,1)}, \theta_{(j,1)}, t_0^{(j,1)}, t_1^{(j,1)})$ ；
s.t. $X_{lower} \leq X \leq X_{upper}$ ；目标函数 $\max \Delta t = g_3(X)$

5.4.2 最优化模型 $GA-4$ 的求解

第四问的求解沿用第二问建立的遗传框架，基础步骤与算法流程均保持不变。下面只列出第四问遗传算法（ GA ）与第二问中不同的关键参数和基本要点。

表 3 模型 $GA-4$ 的相关参数和实数编码对象

实数编码对象	目标函数 g_3 决策变量
种群数目	900
迭代次数	350
线性交叉指数	0.80
基因变异概率	0.25

根据参数，对模型 $GA-4$ 进行求解，最终得到三架飞机各投放一枚烟幕干扰弹的不同投放方案，最终策略设计得到最大有效遮蔽总时间为 8.32s，并将最终设计保存在文件 `result2.xlsx` 中。遗传算法进化过程如图 13 所示。

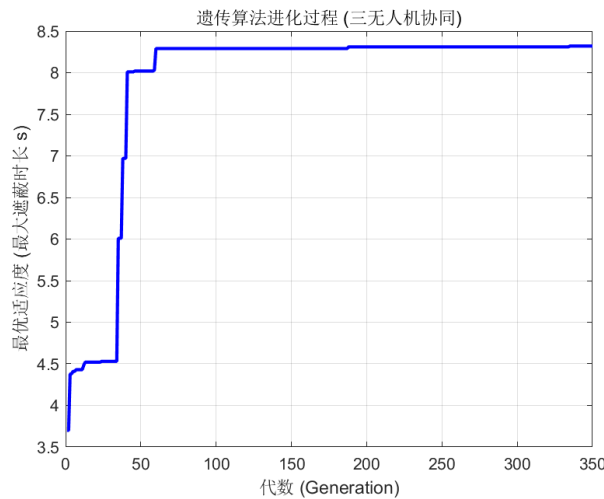


图 13 问题四遗传算法进化过程示意图

5.5 问题五模型的建立与求解

5.5.1 最优化模型 GA-5 的建立

首先，根据题意建立导弹 M_i 的轨迹参数方程和无人机 FY_j 投放第 k 枚烟幕干扰弹球心的轨迹参数方程；接着，根据点到直线的距离公式求出球状烟幕中心点 $Q_{(j,k)}$ 到直线 NM_i 距离；最后，根据题意建立最优化模型 **GA-5**

5.5.1.1 导弹 M_i 的轨迹参数方程和无人机 FY_j 投放第 k 枚烟幕干扰弹球心的轨迹参数方程的建立

记导弹 $M_i (i=1,2,3)$ 坐标为 $M_i (x_{M_i}, y_{M_i}, z_{M_i})$ ；

容易求出：向量 $\overline{OM_i}$ 方向余弦为 $(\cos \alpha_{M_i}, \cos \beta_{M_i}, \cos \gamma_{M_i})$ ，其中：

$$\begin{aligned}\cos \alpha_{M_i} &= \frac{x_{M_i}}{\sqrt{x_{M_i}^2 + y_{M_i}^2 + z_{M_i}^2}} \\ \cos \beta_{M_i} &= \frac{y_{M_i}}{\sqrt{x_{M_i}^2 + y_{M_i}^2 + z_{M_i}^2}} \\ \cos \gamma_{M_i} &= \frac{z_{M_i}}{\sqrt{x_{M_i}^2 + y_{M_i}^2 + z_{M_i}^2}}\end{aligned}$$

$$\text{从而求出导弹 } M_i (i=1,2,3) \text{ 的轨迹参数方程 } \begin{cases} x'_{M_i}(t) = x_{M_i} - v_D t \cos \alpha_{M_i} \\ y'_{M_i}(t) = y_{M_i} - v_D t \cos \beta_{M_i} \\ z'_{M_i}(t) = z_{M_i} - v_D t \cos \gamma_{M_i} \end{cases}$$

无人机 FY_j 投放第 k 枚烟幕干扰弹球心的轨迹参数方程为：

当 $t \in [0, t_0^{(j,k)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(j,k)}(t, v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = x_{F_{(j,k)}} + v_{F_{(j,k)}} \cos \theta_{(j,k)} t \\ y_{(j,k)}(t, v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = y_{F_{(j,k)}} + v_{F_{(j,k)}} \sin \theta_{(j,k)} t \\ z_{(j,k)}(t, v_{F_j}, \theta_j, t_0^{(j,k)}, t_1^{(j,k)}) = z_{F_{(j,k)}} \end{cases}$$

当 $t \in (t_0^{(j,k)}, t_1^{(j,k)}]$ 时，轨迹参数方程为：

$$\begin{cases} x_{(j,k)}(t, v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = x_{F_{(j,k)}} + v_{F_{(j,k)}} \cos \theta_{(j,k)} t \\ y_{(j,k)}(t, v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = y_{F_{(j,k)}} + v_{F_{(j,k)}} \sin \theta_{(j,k)} t \\ z_{(j,k)}(t, v_{F_j}, \theta_j, t_0^{(j,k)}, t_1^{(j,k)}) = z_{F_{(j,k)}} - \int_0^{t-t_0^{(j,k)}} g t dt \end{cases}$$

当 $t \in (t_1^{(j,k)}, t_2^{(j,k)}]$ 时, 轨迹参数方程为:

$$\begin{cases} x_{(j,k)}(t, v_{F(j,k)}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = x_{F(j,k)} + v_{F(j,k)} \cos \theta_{(j,k)} t \\ y_{(j,k)}(t, v_{F(j,k)}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = y_{F(j,k)} + v_{F(j,k)} \sin \theta_{(j,k)} t \\ z_{(j,k)}(t, v_{F(j,k)}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) = z_{F(j,k)} - \int_0^{t-t_1^{(j,k)}} g t dt - v_L (t - t_1^{(j,k)}) \end{cases}$$

其中

$$t_2^{(j,k)} = \frac{z_{F(j,k)} - \int_0^{\Delta t_{(j,k)}} g t dt}{v_L} + t_1^{(j,k)}$$

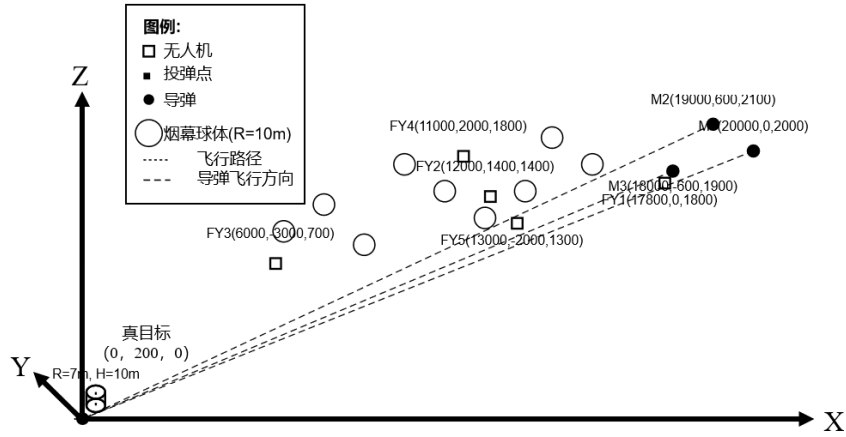


图 12 多无人机多投放弹几何光学遮蔽示意图

5.5.1.2 球状烟雾中心点 $Q_{(j,k)}$ 到直线 NM_i 距离表达式的建立

设烟雾弹球心为 $Q_{(j,k)}(x_{(j,k)}, y_{(j,k)}, z_{(j,k)})$, 真目标圆柱上下圆周上的点 $N(x_N, y_N, z_N)$ 满足:

$$\begin{cases} x_N^2 + y_N^2 \leq R_0^2 \\ z_N = 0, H \end{cases}$$

点 N 与点 $Q_{(j,k)}$ 连线构成向量 $\overline{NQ_{(j,k)}}$, 且

$$\overline{NQ_{(j,k)}} = (x_{(j,k)} - x_N, y_{(j,k)} - y_N, z_{(j,k)} - z_N)$$

点 N 与点 M_i 连线构成向量 $\overline{NM_i}$, 且

$$\overline{NM_i} = (x_{M_i} - x_N, y_{M_i} - y_N, z_{M_i} - z_N)$$

从而得到球状烟雾中心点 $Q_{(j,k)}$ 到直线 NM_i 距离

$$d_{(i,j,k)} = \sqrt{|\overline{NQ_{(j,k)}}|^2 - \left(\frac{\overline{NQ_{(j,k)}} \cdot \overline{NM_i}}{|\overline{NM_i}|} \right)^2}$$

5.5.1.3 最优化模型 GA-5 的建立

圆柱上下圆周上的点和导弹所连线段与烟幕干扰弹起爆后形成的半径 $R=10m$ 球状云团有公共点，只要保证球状烟幕中心点 $Q_{(j,k)}$ 到直线 NM_i 距离不大于球半径 R ，也即：

$$d_{(i,j,k)} \leq R$$

从而

$$f_4\left(t, v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}\right) \leq R$$

设满足 $d_{(i,j,k)} \leq R$ 的若干个时间段为 $\Delta t_i^l (l \in N^+)$ ，则烟幕干扰弹对 M_i 的有效遮蔽时长 Δt_i 为：

$$\Delta t_i = \sum_l \Delta t_i^l$$

我们可以通过遗传算法对矩阵 $X = (v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)})$ 进行合理取值，这样，对于每一个矩阵 X ，都能得到与之对应的有效遮蔽时长 Δt_i ，也就是说我们能得到 Δt_i 关于矩阵 $X = (v_{F_{(j,l)}}, \theta_{(j,l)}, t_0^{(j,l)}, t_1^{(j,l)})$ 的函数，也即：

$$\Delta t_i = g_4(X) = h\left(v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}\right)$$

设 X_{lower} 为矩阵 X 中参数的下确界组成的矩阵， X_{upper} 为矩阵 X 中参数的上确界组成的矩阵，也即：

$$X_{lower} \leq X \leq X_{upper}$$

并满足（忽略单位，且只列出重复部分）

$$\begin{cases} X_{lower} = [70, 0, 0, 0, 70, 0, 1, 0, 70, 0, 3, 0] \\ X_{upper} = [140, 2\pi, 15, 23, 140, 2\pi, 25, 33, 140, 2\pi, 35, 43] \end{cases}$$

因此我们可以建立最优化模型 GA-5，其中决策变量和目标函数分别为：

$$\begin{aligned} X &= (v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}) \\ \text{s.t. } X_{lower} &\leq X \leq X_{upper} \\ \max \Delta t &= g_4\left(v_{F_{(j,k)}}, \theta_{(j,k)}, t_0^{(j,k)}, t_1^{(j,k)}\right) = \sum_{i=1}^3 \Delta t_i \end{aligned}$$

5.5.2 最优化模型 GA-5 的求解

第五问的求解沿用第二问建立的遗传框架，基础步骤与算法流程均保持不变。下面只列出第四问遗传算法（GA）与第二问中不同的关键参数和基本要点。

表 4 模型 5 的相关参数和实数编码对象

实数编码对象	目标函数 g_4 决策变量
种群数目	1500
迭代次数	500
线性交叉指数	0.85
基因变异概率	0.25

使用以上参数求解模型 $GA-5$ 得到五架无人机针对三枚导弹各投放三枚烟雾干扰弹的最终投放方案，最后设计结果得到最大有效遮蔽总时间为 13.85s，并将投放方案保存在文件 result3.xlsx 中。遗传算法进化过程如图 14 所示。

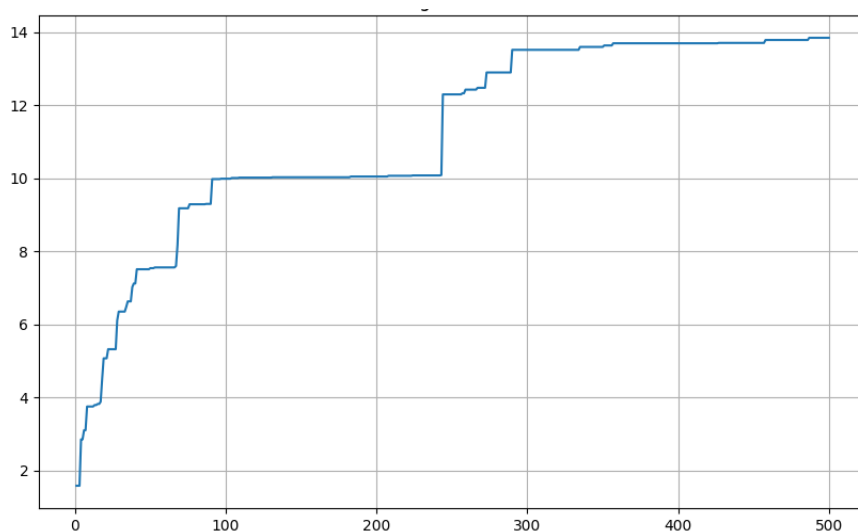


图 14 问题五遗传算法进化过程示意图

六、模型的评价与改进

6.1 模型评价

基于遗传算法无人机协同烟雾遮蔽任务优化，最终最优解为 13.85 秒，并且该值代表着对三枚导弹进行遮蔽的时间综合，通过对相应时间等方面分析，做如下评价：

1. 整体遮盖效率较为低下：总适应度得分仅为 13.85 秒，相对于 15 枚烟雾弹的总潜在遮蔽能力（每枚持续 20 秒）而言，资源利用率极低。在 15 次烟雾弹部署中，仅有 4 次（UAV1 的 1、3 号弹；UAV2 的 2、3 号弹）产生了有效的遮蔽时间，成功率仅为 26.7%。这表明大量的无人机机动和烟雾弹投放并未对最终目标产生贡献，造成了严重的资源浪费。

2. 目标分配严重不均：M3 导弹在整个过程中获得的遮蔽时间为 0。算法找到了一个仅能部分遮蔽 M1 和 M2 的局部最优解，而完全放弃了对 M3 的防御。

这在实际作战中是不可接受的，暴露了当前适应度函数在引导算法全面解决问题上的缺陷。

3. 算法陷入局部最优：可以发现，遗传算法在无人机 3-4-6 中，使用遮蔽率为 0，此时，12 枚烟雾弹并未造成任何遮蔽，故而应当是陷入了局部最优。

6.2 模型改进

针对以上评价中暴露的问题，我们提出以下几个层面的改进方向：

1. 算法层面改进：调整遗传算法参数，扩大种群大小以及迭代次数，增加种群大小以提升多样性，并增加迭代次数给予算法更充分的进化时间；引入自适应变异，在算法迭代后期，当最优适应度长时间停滞不前时，可以动态提高变异率，以帮助算法“跳出”当前的局部最优陷阱，增加寻找新解空间的可能性。

2. 适应度函数改进：引入，惩罚以及奖励机制。增加“保底”惩罚机制：为了解决 M3 完全未被覆盖的问题，可以对适应度函数加入惩罚项。例如，当任何一枚导弹的总遮蔽时间为 0 时，将最终的总适应度分乘以一个极小的惩罚系数甚至直接归零。这将迫使算法从一开始就必须兼顾所有三个目标。引入“协同”奖励机制：为了鼓励形成协同遮蔽，可以增加奖励项。例如，在计算总适应度的基础上，额外增加一个奖励分值，该分值正比于“多枚导弹被同时遮蔽”的时间。

3. 物理模型与约束改进：更改参数的可执行范围，考虑其他因素。

七、参考文献

- [1]王颖,张圆.一种基于平面靶标的圆结构光标定方法[J].红外与激光工程, 2013, 42(S01):5.DOI:10.3969/j.issn.1007-2276.2013.z1.034.
- [2]杜文杰.圆锥面作为辅助面求作球体切平面[J].工程图学学报,2006,27(3):135-138. DOI:10.3969/j.issn.1003-0158.2006.03.024.
- [3]俞健.多视角光栅投影三维重建若干关键问题研究[D].东南大学,2021.
- [4]马永杰,云文霞.遗传算法研究进展[J].计算机应用研究,2012,29(04):1201-1206+1210.
- [5]吉根林.遗传算法研究综述[J].计算机应用与软件,2004,(02):69-73.
- [6]包子阳,余继周.智能优化算法及其 MATLAB 实例[M].电子工业出版社:201608:200.

附录

附录
支撑材料清单
1. 问题一求解代码 a1.m
2. 问题二求解代码 a2.m
3. 问题三求解代码 a3.m
4. 问题四求解代码 a4.m
5. 问题五求解代码 a5.py
6. 问题一结果图像 a1.png
7. 问题二结果图像 a2.png
8. 问题三结果图像 a3.png
9. 问题四结果图像 a4.png
10. 问题五结果图像 a5.png
11. AI 工具使用详情.pdf
12. GA 算法流程图.docx
13. GA 算法流程.drawio
14. 问题三求解结果 result1.xlsx
15. 问题四求解结果 result2.xlsx
16. 问题五求解结果 result3.xlsx
17. 导弹烟幕遮蔽原理侧视图+R.png
18. 导弹烟幕遮蔽原理俯视图+R.png
19. 导弹烟幕遮蔽原理图.png
20. 问题一二过程图 几何光学导弹烟幕遮蔽坐标示意图 1.png
21. 问题三过程图 几何光学导弹烟幕遮蔽坐标示意图 3.png
22. 问题四过程图 几何光学导弹烟幕遮蔽坐标示意图 4.png
23. 问题五过程图 几何光学导弹烟幕遮蔽坐标示意图 5.png
24. 问题一 二 开始遮蔽示意图.png
25. 问题一 二 结束遮蔽示意图.png
26. 遗传流程图.jfif
27. 铸锥证明示意图 1.png
28. 使用模型简介图 两模型代号网络图.png
29. 实地使用示意图 图片 1.png
30. 问题三结果.txt
31. 问题三图像.fig
32. 问题三图像.png
33. 问题五结果.docx

34.问题五图像.fig

附录 2

目录

1. 问题一求解（MATLAB 源码）-附录 2
2. 问题一求解结果-附录 3
3. 问题二求解（MATLAB 源码）-附录 4
4. 问题二求解结果-附录 5
5. 问题三求解（MATLAB 源码）-附录 6
6. 问题三求解结果-附录 7
7. 问题四求解（MATLAB 源码）-附录 8
8. 问题四求解结果-附录 9
9. 问题五求解（PYTHON 源码）-附录 10
- 10.问题五求解结果-附录 11

附录 2

问题一求解（MATLAB 源码）

```

1.  %初始位置 v
2.  M0 = [20000, 0, 2000]; %导初坐标
3.  v_m = 300; %导速度
4.  A0 = [17800, 0, 1800]; %飞机初坐标
5.  v_a = 120; %飞机速度
6.  O = [0, 0, 0]; %原点
7.
8.  % 真实目标
9.  T_c = [0, 200, 0]; %圆柱中心
10. r_t = 7; %圆柱半径
11. h_t = 10; %圆柱高度
12.
13. % 干扰弹和烟雾
14. t1 = 1.5; %投放时间
15. t_delay = 3.6; %延时起爆时间
16. v_down = 3; %烟雾云下沉速度
17. r_s = 10; %烟雾有效遮蔽半径
18. t_s_e_g = 20; %烟雾遮蔽时长
19. g = 9.8;

```

```

20.
21. %模拟 时间
22. t_s_g = 0;
23. t_e_g = 40;          %总时长
24. dt = 0.0001;        %步长
25. t_vec = t_s_g:dt:t_e_g;%时间向量
26.
27. %save
28. is_obscured = false(size(t_vec)); %存储每个步长 遮蔽方式
29.
30.
31. %计算 干扰弹 起爆点
32. t_deploy = t1;
33. t_exp = t1 + t_delay;
34.
35. pptd_g = getPlanePos_g(t_deploy, A0, v_a);
36. %烟雾弹 平抛运动
37. smoke_exp_pos = [pptd_g(1) - v_a * t_delay, ...
38.                  pptd_g(2), ...
39.                  pptd_g(3) - 0.5 * g * t_delay^2];
40.
41. for i = 1:length(t_vec)
42.     t = t_vec(i);
43.
44.     %是否 在烟雾有效遮蔽 内（时间）
45.     if t >= t_exp && t <= t_exp + t_s_e_g
46.         missile_pos_ggg = gmp_g(t, M0, v_m, 0);
47.
48.         %烟雾球中心 移动方程
49.         smoke_pos = smoke_exp_pos + [0, 0, -
v_down * (t - t_exp)];
50.
51.         %真实目标 是否 完全遮蔽
52.         is_fully_obscured_now_ggg = true;
53.
54.         %采样 真实目标
55.         num_samples = 10000;
56.         theta_vec = linspace(0, 2*pi, num_samples+1);
57.         theta_vec(end) = [];
58.
59.         %检查 下底圆周

```

```

60.         for j = 1:num_samples
61.             x_t = T_c(1) + r_t * cos(theta_vec(j));
62.             y_t = T_c(2) + r_t * sin(theta_vec(j));
63.             z_t = T_c(3) - h_t/2;
64.             target_point = [x_t, y_t, z_t];
65.             if ~cpsi_g(missile_pos_ggg, target_point, smoke_pos, r_s)
66.                 is_fully_obsured_now_ggg = false;
67.                 break;
68.             end
69.         end
70.
71.         if ~is_fully_obsured_now_ggg
72.             is_obsured(i) = false;
73.             continue;
74.         end
75.
76.         % 检查 上底圆周
77.         for j = 1:num_samples
78.             x_t = T_c(1) + r_t * cos(theta_vec(j));
79.             y_t = T_c(2) + r_t * sin(theta_vec(j));
80.             z_t = T_c(3) + h_t/2;
81.             target_point = [x_t, y_t, z_t];
82.             if ~cpsi_g(missile_pos_ggg, target_point, smoke_pos, r_s)
83.                 is_fully_obsured_now_ggg = false;
84.                 break;
85.             end
86.         end
87.
88.         is_obsured(i) = is_fully_obsured_now_ggg;
89.     end
90. end
91.
92. %被遮蔽 时间点位
93. if any(is_obsured)
94.     %总遮蔽 时长
95.     total_ggg = sum(is_obsured) * dt;
96.     fprintf('总遮蔽时长为: %.4f s. \n\n', total_ggg);
97.
98.     %遮蔽时段

```

```

99.      fprintf('真实目标被完全遮蔽的时间段如下: \n');
100.
101.      diff_obsured_g = diff([0; is_obsured(:); 0]);
102.      start_indices = find(diff_obsured_g == 1);
103.      end_indices = find(diff_obsured_g == -1) - 1;
104.
105.      for k = 1:length(start_indices)
106.          start_time = t_vec(start_indices(k));
107.          end_time = t_vec(end_indices(k));
108.          fprintf(' - 从 %.4f s 到 %.4f s\n', start_time, e
nd_time);
109.      end
110.
111.  else
112.      fprintf('真实目标未被完全遮蔽.\n');
113.  end
114.
115.
116. %局部函数
117.
118. function pos = gmp_g(t, M0, v_m, 0)
119.     %导弹 位置
120.     dir_vec = (0 - M0) / norm(0 - M0);
121.     pos = M0 + v_m * t * dir_vec;
122. end
123.
124. function pos = getPlanePos_g(t, A0, v_a)
125.     %飞机位置 按照之前的运动方向
126.     pos = A0 + [-v_a * t, 0, 0];
127. end
128.
129. function intersect_ggg = cpsi_g(P, Q, C, r)
130.     %检测是否 遮蔽 判断方式, 连接线段, 是否与球有交点
131.     vec_ray = Q - P;
132.     vec_PC = C - P;
133.
134.     t = dot(vec_PC, vec_ray) / dot(vec_ray, vec_ray);
135.
136.     if t < 0
137.         intersect_ggg = false;
138.         return;

```



```

139.     end
140.
141.     dist_sq = norm(vec_PC)^2 - t^2 * norm(vec_ray)^2;
142.
143.     if dist_sq <= r^2
144.         intersect_ggg = true;
145.     else
146.         intersect_ggg = false;
147.     end
148. end

```

附录 3

问题一求解结果

总遮蔽时长为：1.3387 s。

真实目标被完全遮蔽的时间段如下：

- 从 8.0799 s 到 9.4185 s

附录 4

问题二求解（MATLAB 源码）

```

1.  clear; clc;
2.  tic; %计时
3.
4.  global M0 v_m O T_c r_t h_t v_down r_s t_e_ggg dt g A0 v_a
; % t_smoke_effective -> t_e_ggg
5.
6.
7.  M0 = [20000, 0, 2000]; %导弹 初 始 坐标
8.  v_m = 300; %导弹 速度
9.  A0 = [17800, 0, 1800];
10. v_a = 120; %无人机 投放前 速度 假设值
11. O = [0, 0, 0];
12.
13. %真实目标
14. T_c = [0, 200, 0];
15. r_t = 7;
16. h_t = 10;
17.

```

```

18. %干扰弹和烟雾
19. v_down = 3; %烟下沉速度
20. r_s = 10; %半径
21. t_e_ggg = 20; %遮蔽时长
22. g = 9.8;
23.
24. % 模拟时间参数
25. t_start = 0;
26. t_end = 40; % 模拟总时长
27. dt = 0.01; % 时间步长
28. t_f_ggg = t_start:dt:t_end;
29.
30. fprintf('进行预计算以加速优化过程...\n');
31.
32. m_t_ggg = g_p_ggg(t_f_ggg, M0, v_m, 0);
33.
34. n_s_ggg = 100;
35. t_v_ggg = linspace(0, 2*pi, n_s_ggg);
36. t_b_ggg = [T_c(1) + r_t * cos(t_v_ggg); ...
37.           T_c(2) + r_t * sin(t_v_ggg); ...
38.           repmat(T_c(3) - h_t/2, 1, n_s_ggg)];
39. t_p_ggg = [T_c(1) + r_t * sin(t_v_ggg); ...
40.           T_c(2) + r_t * cos(t_v_ggg); ...
41.           repmat(T_c(3) + h_t/2, 1, n_s_ggg)];
42. a_p_ggg = [t_b_ggg, t_p_ggg];
43.
44.
45.
46. fprintf('开始进行 FY1 无人机干扰方案优化 (遗传算法)...\n');
47.
48.
49. p_s_ggg = 600; % 种群 大小
50. n_g_ggg = 250; % 迭代 代数
51. c_r_ggg = 0.8; % 交叉 概率
52. m_r_ggg = 0.16; % 变异 概率
53. e_c_ggg = 2; % 精英个 体 数量
54. t_o_ggg = 3; % 锦标赛 选择 规模
55.
56. bounds.lower = [70, 0, 0, 0];
57. bounds.upper = [140, 20, 2*pi, 8];
58.

```

```

59. p_p_ggg = zeros(p_s_ggg, 4);
60. for i = 1:4
61.     p_p_ggg(:, i) = bounds.lower(i) + (bounds.upper(i) - b
ounds.lower(i)) * rand(p_s_ggg, 1);
62. end
63.
64. b_h_ggg = zeros(n_g_ggg, 1);
65. g_f_ggg = -1;
66. g_i_ggg = zeros(1, 4);
67.
68. for gen = 1:n_g_ggg
69.     fitness = zeros(p_s_ggg, 1);
70.     d_i_ggg = (0 - A0) / norm(0 - A0);
71.
72.     for i = 1:p_s_ggg
73.         individual = p_p_ggg(i, :);
74.         v_fy1 = individual(1);
75.         t_deploy = individual(2);
76.         theta_yaw = individual(3);
77.         t_delay = individual(4);
78.
79.         d_p_ggg = A0 + v_a * t_deploy * d_i_ggg;
80.         d_v_ggg = [v_fy1 * cos(theta_yaw), v_fy1 * sin(the
ta_yaw), 0];
81.
82.         fitness(i) = c_o_ggg(t_deploy, d_p_ggg, d_v_ggg, t
_delay, ...
83.                               m_t_ggg, a_p_ggg, t_f_ggg);
84.     end
85.
86.     [m_c_ggg, idx] = max(fitness);
87.     if m_c_ggg > g_f_ggg
88.         g_f_ggg = m_c_ggg;
89.         g_i_ggg = p_p_ggg(idx, :);
90.     end
91.     b_h_ggg(gen) = g_f_ggg;
92.
93.     fprintf('第 %d 代: 当前最优遮蔽时间 = %.4f s, 全局最
优 = %.4f s\n', gen, m_c_ggg, g_f_ggg);
94.
95.     n_p_ggg = zeros(size(p_p_ggg));
96.

```

```

97.     [~, s_i_ggg] = sort(fitness, 'descend');
98.     n_p_ggg(1:e_c_ggg, :) = p_p_ggg(s_i_ggg(1:e_c_ggg), :)
;
99.
100.    for i = (e_c_ggg + 1):2:p_s_ggg
101.        parent1_idx = s_t_ggg(fitness, t_o_ggg);
102.        parent2_idx = s_t_ggg(fitness, t_o_ggg);
103.        parent1 = p_p_ggg(parent1_idx, :);
104.        parent2 = p_p_ggg(parent2_idx, :);
105.
106.        child1 = parent1;
107.        child2 = parent2;
108.        if rand < c_r_ggg
109.            alpha = rand;
110.            child1 = alpha * parent1 + (1 - alpha) * paren
t2;
111.            child2 = alpha * parent2 + (1 - alpha) * paren
t1;
112.        end
113.
114.        child1 = m_m_ggg(child1, m_r_ggg, bounds);
115.        child2 = m_m_ggg(child2, m_r_ggg, bounds);
116.
117.        n_p_ggg(i, :) = child1;
118.        if i+1 <= p_s_ggg
119.            n_p_ggg(i+1, :) = child2;
120.        end
121.    end
122.    p_p_ggg = n_p_ggg;
123. end
124.
125.
126. fprintf('\n 遗传算法优化完成! \n');
127.
128. b_v_ggg = g_i_ggg(1);
129. b_t_ggg = g_i_ggg(2);
130. b_a_ggg = g_i_ggg(3);
131. b_d_ggg = g_i_ggg(4);
132. m_o_ggg = g_f_ggg;
133.
134. if m_o_ggg > 0
135.     b_i_ggg = (0 - A0) / norm(0 - A0);

```

```

136.     b_p_ggg = A0 + v_a * b_t_ggg * b_i_ggg;
137.     b_e_ggg = [b_v_ggg * cos(b_a_ggg), b_v_ggg * sin(b_a_g
gg), 0];
138.
139.     b_x_ggg = b_p_ggg + b_e_ggg * b_d_ggg + [0, 0, -
0.5 * g * b_d_ggg^2];
140.
141.     fprintf('最优无人机速度: %.2f m/s\n', b_v_ggg);
142.     fprintf('最优投放时间: %.2f s\n', b_t_ggg);
143.     fprintf('最优延迟起爆时间: %.2f s\n', b_d_ggg);
144.     fprintf('最优方向 (偏航
角): %.2f rad (%.2f 度)\n', b_a_ggg, rad2deg(b_a_ggg));
145.     fprintf('最大遮蔽时长: %.2f s\n', m_o_ggg);
146.     fprintf('最优投放点: (%.2f, %.2f, %.2f)\n', b_p_ggg);
147.     fprintf('最优起爆点: (%.2f, %.2f, %.2f)\n', b_x_ggg);
148. else
149.     fprintf('在搜索范围内未找到有效的遮蔽方案。 \n');
150. end
151. toc;
152.
153. figure;
154. plot(1:n_g_ggg, b_h_ggg, 'b-', 'LineWidth', 2);
155. title('遗传算法进化过程');
156. xlabel('代数 (Generation)');
157. ylabel('最优适应度 (最大遮蔽时长 s)');
158. grid on;
159.
160. filename = 'a2.png';
161. saveas(gcf, filename);
162.
163. function o_t_ggg = c_o_ggg(t_d_ggg, d_p_ggg, d_v_ggg, l_d_
ggg, m_t_ggg, a_p_ggg, t_f_ggg)
164.     global v_down r_s t_e_ggg dt g;
165.     t_exp = t_d_ggg + l_d_ggg;
166.     s_e_ggg = t_exp + t_e_ggg;
167.     start_idx = floor(t_exp / dt) + 1;
168.     end_idx = min(floor(s_e_ggg / dt) + 1, length(t_f_ggg)
);
169.     if start_idx > end_idx
170.         o_t_ggg = 0;
171.         return;
172.     end

```

```

173.     o_s_ggg = 0;
174.     s_p_ggg = d_p_ggg + d_v_ggg*l_d_ggg + [0, 0, -
0.5*g*l_d_ggg^2];
175.     for i = start_idx:end_idx
176.         t = t_f_ggg(i);
177.         missile_pos = m_t_ggg(i, :);
178.         e_t_ggg = t - t_exp;
179.         s_o_ggg = s_p_ggg + [0, 0, -v_down * e_t_ggg];
180.         if c_i_ggg(missile_pos, a_p_ggg, s_o_ggg, r_s)
181.             o_s_ggg = o_s_ggg + 1;
182.         end
183.     end
184.     o_t_ggg = o_s_ggg * dt;
185. end

186.
187. function p_m_ggg = g_p_ggg(t_v_ggg, M0, v_m, 0)
188.     d_v_ggg = (0 - M0) / norm(0 - M0);
189.     p_m_ggg = M0 + t_v_ggg' * (v_m * d_v_ggg);
190. end

191.
192. function i_f_ggg = c_i_ggg(P, q_m_ggg, C, r)
193.     v_r_ggg = q_m_ggg' - P;
194.     v_p_ggg = C - P;
195.     t = dot(v_r_ggg, repmat(v_p_ggg, size(v_r_ggg, 1), 1),
2) ./ dot(v_r_ggg, v_r_ggg, 2);
196.     v_t_ggg = t > 0;
197.     if ~any(v_t_ggg)
198.         i_f_ggg = false;
199.         return;
200.     end
201.     d_s_ggg = norm(v_p_ggg)^2 - (t.^2) .* dot(v_r_ggg, v_r
_ggg, 2);
202.     intersect = (d_s_ggg <= r^2);
203.     i_f_ggg = all(intersect(v_t_ggg));
204. end

205.
206. function s_e_ggg = s_t_ggg(fitness, t_o_ggg)
207.     p_s_ggg = length(fitness);
208.     indices = randi(p_s_ggg, 1, t_o_ggg);
209.     t_i_ggg = fitness(indices);
210.     [~, b_t_ggg] = max(t_i_ggg);
211.     s_e_ggg = indices(b_t_ggg);

```

```

212. end
213.
214. function individual = m_m_ggg(individual, m_r_ggg, bounds)
215.     for i = 1:length(individual)
216.         if rand < m_r_ggg
217.             range = bounds.upper(i) - bounds.lower(i);
218.             m_v_ggg = (range * 0.1) * randn;
219.             individual(i) = individual(i) + m_v_ggg;
220.             individual(i) = max(individual(i), bounds.lower
r(i));
221.             individual(i) = min(individual(i), bounds.uppe
r(i));
222.         end
223.     end
224. end

```

附录 5

问题二求解结果

投放时间:0.00 s
 投放后无人机速度:138.00 m/s
 投放后无人机方向(偏航角):0.12 rad(6.78 度)
 延迟起爆时间:0.75 s
 投放点:(17799.92, 0.00, 1799.99)起爆点:(17903.37, 12.30, 1797.20)
 该弹单独有效遮蔽时长:4.54s

附录 6

问题三求解 (MATLAB 源码)

```

1. clear; clc;
2. tic; %计时
3.
4. global M0 v_m O T_c r_t h_t v_down r_s t_e_ggg dt g A0 v_a
;
5.
6. M0 = [20000, 0, 2000]; %导弹 初 始 坐标

```

```

7.   v_m = 300;           %导弹   速度
8.   A0 = [17800, 0, 1800];
9.   v_a = 120;           %无人机   投放前   速度   假设值
10.  O = [0, 0, 0];
11.
12.  %真实目标
13.  T_c = [0, 200, 0];
14.  r_t = 7;
15.  h_t = 10;
16.
17.  %干扰弹和烟雾
18.  v_down = 3;           %烟下沉速度
19.  r_s = 10;             %半径
20.  t_e_ggg = 20;         %遮蔽时长
21.  g = 9.8;
22.
23.  %模拟时间参数
24.  t_start = 0;
25.  t_end = 60;            %模拟总时长
26.  dt = 0.01;            %时间步长
27.  t_f_ggg = t_start:dt:t_end;
28.
29.  fprintf('进行预计算以加速优化过程...\n');
30.  m_t_ggg = g_m_ggg(t_f_ggg, M0, v_m, 0);
31.  n_s_ggg = 50;
32.  t_v_ggg = linspace(0, 2*pi, n_s_ggg);
33.  t_b_ggg = [T_c(1) + r_t * cos(t_v_ggg); T_c(2) + r_t * sin
(t_v_ggg); repmat(T_c(3) - h_t/2, 1, n_s_ggg)];
34.  t_p_ggg = [T_c(1) + r_t * sin(t_v_ggg); T_c(2) + r_t * cos
(t_v_ggg); repmat(T_c(3) + h_t/2, 1, n_s_ggg)];
35.  a_p_ggg = [t_b_ggg, t_p_ggg];
36.
37.  fprintf('开始进行三枚烟幕弹干扰方案优化 (遗传算法)...\n');
38.
39.  p_s_ggg = 600;         %种群   大小
40.  n_g_ggg = 350;         %迭代   代数
41.  n_e_ggg = 12;          %基因   数量
42.  c_r_ggg = 0.86;        %交叉   概率
43.  m_r_ggg = 0.17;        %变异   概率
44.  e_c_ggg = 2;           %精英   个体   数量
45.  t_s_ggg = 3;           %锦标赛 选择   规模

```



```

46.
47.
48.  bounds.lower = [ 0, 70, 0, 0, 1, 70, 0, 0,
2, 70, 0, 0];
49.  bounds.upper = [10, 140, 2*pi, 8, 25, 140, 2*pi, 8,
35, 140, 2*pi, 8];
50.
51.  p_p_ggg = zeros(p_s_ggg, n_e_ggg);
52.  for i = 1:n_e_ggg
53.      p_p_ggg(:, i) = bounds.lower(i) + (bounds.upper(i) - b
ounds.lower(i)) * rand(p_s_ggg, 1);
54.  end
55.
56.  b_h_ggg = zeros(n_g_ggg, 1);
57.  g_f_ggg = -1;
58.  g_i_ggg = zeros(1, n_e_ggg);
59.  d_i_ggg = (0 - A0) / norm(0 - A0);
60.
61.  for gen = 1:n_g_ggg
62.      fitness = zeros(p_s_ggg, 1);
63.      for i = 1:p_s_ggg
64.          ind = p_p_ggg(i, :);
65.          t_d1_ggg = ind(1); v_f1_ggg = ind(2); t_y1_ggg =
ind(3); t_l1_ggg = ind(4); % t_deploy1, v_fy1, theta_yaw1, t_delay1
66.          d_t2_ggg = ind(5); v_f2_ggg = ind(6); t_y2_ggg =
ind(7); t_l2_ggg = ind(8); % delta_t2, v_fy2, theta_yaw2, t_delay2
67.          d_t3_ggg = ind(9); v_f3_ggg = ind(10); t_y3_ggg =
ind(11); t_l3_ggg = ind(12); % delta_t3, v_fy3, theta_yaw3, t_delay3
68.
69.          t_d2_ggg = t_d1_ggg + 1 + d_t2_ggg;
70.          t_d3_ggg = t_d2_ggg + 1 + d_t3_ggg;
71.
72.          if t_d3_ggg > t_end
73.              fitness(i) = 0;
74.              continue;
75.          end
76.
77.          a_p1_ggg = A0 + v_a * t_d1_ggg * d_i_ggg;
78.          d_v1_ggg = [v_f1_ggg * cos(t_y1_ggg), v_f1_ggg * s
in(t_y1_ggg), 0];
79.          a_p2_ggg = a_p1_ggg + d_v1_ggg * (t_d2_ggg - t_d1_
ggg);

```

```

80.         d_v2_ggg = [v_f2_ggg * cos(t_y2_ggg), v_f2_ggg * s
in(t_y2_ggg), 0];
81.         a_p3_ggg = a_p2_ggg + d_v2_ggg * (t_d3_ggg - t_d2_
ggg);
82.         d_v3_ggg = [v_f3_ggg * cos(t_y3_ggg), v_f3_ggg * s
in(t_y3_ggg), 0];
83.
84.         params.grenade1 = struct('t_deploy', t_d1_ggg, 'de
ploy_pos', a_p1_ggg, 'deploy_v_vec', d_v1_ggg, 't_delay', t_l1_ggg,
'v_fy', v_f1_ggg, 'theta_yaw', t_y1_ggg);
85.         params.grenade2 = struct('t_deploy', t_d2_ggg, 'de
ploy_pos', a_p2_ggg, 'deploy_v_vec', d_v2_ggg, 't_delay', t_l2_ggg,
'v_fy', v_f2_ggg, 'theta_yaw', t_y2_ggg);
86.         params.grenade3 = struct('t_deploy', t_d3_ggg, 'de
ploy_pos', a_p3_ggg, 'deploy_v_vec', d_v3_ggg, 't_delay', t_l3_ggg,
'v_fy', v_f3_ggg, 'theta_yaw', t_y3_ggg);
87.
88.         fitness(i) = c_t_ggg(params, m_t_ggg, a_p_ggg, t_f
_ggg);
89.     end
90.
91.     [m_c_ggg, idx] = max(fitness);
92.     if m_c_ggg > g_f_ggg
93.         g_f_ggg = m_c_ggg;
94.         g_i_ggg = p_p_ggg(idx, :);
95.     end
96.     b_h_ggg(gen) = g_f_ggg;
97.     fprintf('第 %d 代: 当前最优遮蔽时间 = %.4f s, 全局最
优 = %.4f s\n', gen, m_c_ggg, g_f_ggg);
98.
99.     n_p_ggg = zeros(size(p_p_ggg));
100.    [~, s_i_ggg] = sort(fitness, 'descend');
101.    n_p_ggg(1:e_c_ggg, :) = p_p_ggg(s_i_ggg(1:e_c_ggg), :)
;
102.
103.    for i = (e_c_ggg + 1):2:p_s_ggg
104.        parent1 = p_p_ggg(s_t_ggg(fitness, t_s_ggg), :);
105.        parent2 = p_p_ggg(s_t_ggg(fitness, t_s_ggg), :);
106.
107.        child1 = parent1; child2 = parent2;
108.        if rand < c_r_ggg
109.            alpha = rand;
110.            child1 = alpha * parent1 + (1 - alpha) * paren
t2;

```

```

111.         child2 = alpha * parent2 + (1 - alpha) * paren
t1;
112.         end
113.
114.         n_p_ggg(i, :) = m_m_ggg(child1, m_r_ggg, bounds);
% mutate -> m_m_ggg
115.         if i+1 <= p_s_ggg
116.             n_p_ggg(i+1, :) = m_m_ggg(child2, m_r_ggg, boun
ds);
117.         end
118.     end
119.     p_p_ggg = n_p_ggg;
120. end
121.
122. fprintf('\n 遗传算法优化完成! \n');
123.
124. best_ind = g_i_ggg;
125. p_f_ggg.grenade1.t_deploy = best_ind(1); p_f_ggg.grenade1
.v_fy = best_ind(2); p_f_ggg.grenade1.theta_yaw = best_ind(3); p_f_
ggg.grenade1.t_delay = best_ind(4);
126. d_t2_ggg = best_ind(5); p_f_ggg.grenade2.v_fy = best_ind(
6); p_f_ggg.grenade2.theta_yaw = best_ind(7); p_f_ggg.grenade2.t_de
lay = best_ind(8);
127. d_t3_ggg = best_ind(9); p_f_ggg.grenade3.v_fy = best_ind(
10);p_f_ggg.grenade3.theta_yaw = best_ind(11);p_f_ggg.grenade3.t_de
lay = best_ind(12);
128.
129. p_f_ggg.grenade2.t_deploy = p_f_ggg.grenade1.t_deploy + 1
+ d_t2_ggg;
130. p_f_ggg.grenade3.t_deploy = p_f_ggg.grenade2.t_deploy + 1
+ d_t3_ggg;
131.
132. g1 = p_f_ggg.grenade1;
133. g2 = p_f_ggg.grenade2;
134. g3 = p_f_ggg.grenade3;
135.
136. g1.deploy_pos = A0 + v_a * g1.t_deploy * d_i_ggg;
137. g1.deploy_v_vec = [g1.v_fy * cos(g1.theta_yaw), g1.v_fy *
sin(g1.theta_yaw), 0];
138. g2.deploy_pos = g1.deploy_pos + g1.deploy_v_vec * (g2.t_de
ploy - g1.t_deploy);
139. g2.deploy_v_vec = [g2.v_fy * cos(g2.theta_yaw), g2.v_fy *
sin(g2.theta_yaw), 0];

```

```

140. g3.deploy_pos = g2.deploy_pos + g2.deploy_v_vec * (g3.t_deploy - g2.t_deploy);
141. g3.deploy_v_vec = [g3.v_fy * cos(g3.theta_yaw), g3.v_fy * sin(g3.theta_yaw), 0];
142.
143. e_p1_ggg = g1.deploy_pos + g1.deploy_v_vec * g1.t_delay + [0, 0, -0.5 * g * g1.t_delay^2];
144. e_p2_ggg = g2.deploy_pos + g2.deploy_v_vec * g2.t_delay + [0, 0, -0.5 * g * g2.t_delay^2];
145. e_p3_ggg = g3.deploy_pos + g3.deploy_v_vec * g3.t_delay + [0, 0, -0.5 * g * g3.t_delay^2];
146.
147. fprintf('正在计算各烟雾弹的单独贡献...\n');
148. o_t_1_ggg = c_i_ggg(g1, m_t_ggg, a_p_ggg, t_f_ggg);
149. o_t_2_ggg = c_i_ggg(g2, m_t_ggg, a_p_ggg, t_f_ggg);
150. o_t_3_ggg = c_i_ggg(g3, m_t_ggg, a_p_ggg, t_f_ggg);
151. fprintf('计算完成。 \n\n');
152.
153. fprintf('最大总遮蔽时长: %.2f s\n\n', g_f_ggg);
154. fprintf('--- 烟雾弹 1 ---\n');
155. fprintf('投放时间: %.2f s\n', g1.t_deploy);
156. fprintf('投放后无人机速度: %.2f m/s\n', g1.v_fy);
157. fprintf('投放后无人机方向 (偏航角): %.2f rad (%.2f 度)\n', g1.theta_yaw, rad2deg(g1.theta_yaw));
158. fprintf('延迟起爆时间: %.2f s\n', g1.t_delay);
159. fprintf('投放点: (%.2f, %.2f, %.2f)\n', g1.deploy_pos);
160. fprintf('起爆点: (%.2f, %.2f, %.2f)\n', e_p1_ggg);
161. fprintf('** 该弹单独有效遮蔽时长: %.2f s **\n\n', o_t_1_ggg);
162.
163. fprintf('--- 烟雾弹 2 ---\n');
164. fprintf('投放时间: %.2f s\n', g2.t_deploy);
165. fprintf('投放后无人机速度: %.2f m/s\n', g2.v_fy);
166. fprintf('投放后无人机方向 (偏航角): %.2f rad (%.2f 度)\n', g2.theta_yaw, rad2deg(g2.theta_yaw));
167. fprintf('延迟起爆时间: %.2f s\n', g2.t_delay);
168. fprintf('投放点: (%.2f, %.2f, %.2f)\n', g2.deploy_pos);
169. fprintf('起爆点: (%.2f, %.2f, %.2f)\n', e_p2_ggg);
170. fprintf('** 该弹单独有效遮蔽时长: %.2f s **\n\n', o_t_2_ggg);
171.
172. fprintf('--- 烟雾弹 3 ---\n');

```

```

173. fprintf('投放时间: %.2f s\n', g3.t_deploy);
174. fprintf('投放后无人机速度: %.2f m/s\n', g3.v_fy);
175. fprintf('投放后无人机方向 (偏航
角): %.2f rad (%.2f 度)\n', g3.theta_yaw, rad2deg(g3.theta_yaw));
176. fprintf('延迟起爆时间: %.2f s\n', g3.t_delay);
177. fprintf('投放点: (%.2f, %.2f, %.2f)\n', g3.deploy_pos);
178. fprintf('起爆点: (%.2f, %.2f, %.2f)\n', e_p3_ggg);
179. fprintf('** 该弹单独有效遮蔽时
长: %.2f s **\n\n', o_t_3_ggg);
180.
181. toc;
182.
183. figure;
184. plot(1:n_g_ggg, b_h_ggg, 'b-', 'LineWidth', 2);
185. title('遗传算法进化过程 (三枚烟幕弹)');
186. xlabel('代数 (Generation)');
187. ylabel('最优适应度 (最大遮蔽时长 s)');
188. grid on;
189.
190. filename = 'a3.png';
191. saveas(gcf, filename);
192.
193.
194.
195. function o_t_ggg = c_t_ggg(params, m_t_ggg, a_p_ggg, t_f_g
gg)
196.     global v_down r_s t_e_ggg dt g;
197.     g1 = params.grenade1; g2 = params.grenade2; g3 = param
s.grenade3;
198.     t_e1_ggg = g1.t_deploy + g1.t_delay; t_e2_ggg = g2.t_d
eploy + g2.t_delay; t_e3_ggg = g3.t_deploy + g3.t_delay;
199.     s_p1_ggg = g1.deploy_pos + g1.deploy_v_vec*g1.t_delay
+ [0, 0, -0.5*g*g1.t_delay^2];
200.     s_p2_ggg = g2.deploy_pos + g2.deploy_v_vec*g2.t_delay
+ [0, 0, -0.5*g*g2.t_delay^2];
201.     s_p3_ggg = g3.deploy_pos + g3.deploy_v_vec*g3.t_delay
+ [0, 0, -0.5*g*g3.t_delay^2];
202.     t_s1_ggg = t_e1_ggg; t_d1_ggg = t_e1_ggg + t_e_ggg;
203.     t_s2_ggg = t_e2_ggg; t_d2_ggg = t_e2_ggg + t_e_ggg;
204.     t_s3_ggg = t_e3_ggg; t_d3_ggg = t_e3_ggg + t_e_ggg;
205.     m_s_ggg = min([t_s1_ggg, t_s2_ggg, t_s3_ggg]);
206.     m_e_ggg = max([t_d1_ggg, t_d2_ggg, t_d3_ggg]);

```

```

207.     start_idx = floor(m_s_ggg / dt) + 1;
208.     end_idx = min(floor(m_e_ggg / dt) + 1, length(t_f_ggg)
);
209.     if start_idx > end_idx; o_t_ggg = 0; return; end
210.     o_s_ggg = 0;
211.     for i = start_idx:end_idx
212.         t = t_f_ggg(i); missile_pos = m_t_ggg(i, :);
213.         a_c_ggg = [];
214.         if t >= t_s1_ggg && t <= t_d1_ggg
215.             a_c_ggg = [a_c_ggg; s_p1_ggg + [0, 0, -
v_down * (t - t_e1_ggg)]];
216.         end
217.         if t >= t_s2_ggg && t <= t_d2_ggg
218.             a_c_ggg = [a_c_ggg; s_p2_ggg + [0, 0, -
v_down * (t - t_e2_ggg)]];
219.         end
220.         if t >= t_s3_ggg && t <= t_d3_ggg
221.             a_c_ggg = [a_c_ggg; s_p3_ggg + [0, 0, -
v_down * (t - t_e3_ggg)]];
222.         end
223.         if isempty(a_c_ggg); continue; end
224.         if i_f_ggg(missile_pos, a_p_ggg, a_c_ggg, r_s)
225.             o_s_ggg = o_s_ggg + 1;
226.         end
227.     end
228.     o_t_ggg = o_s_ggg * dt;
229. end
230.
231. function o_t_ggg = c_i_ggg(g_p_ggg, m_t_ggg, a_p_ggg, t_f_
ggg)
232.     global v_down r_s t_e_ggg dt g;
233.     g1 = g_p_ggg;
234.     t_e1_ggg = g1.t_deploy + g1.t_delay;
235.     s_p1_ggg = g1.deploy_pos + g1.deploy_v_vec*g1.t_delay
+ [0, 0, -0.5*g*g1.t_delay^2];
236.     t_s1_ggg = t_e1_ggg;
237.     t_d1_ggg = t_e1_ggg + t_e_ggg;
238.     start_idx = floor(t_s1_ggg / dt) + 1;
239.     end_idx = min(floor(t_d1_ggg / dt) + 1, length(t_f_ggg
));
240.     if start_idx > end_idx; o_t_ggg = 0; return; end
241.     o_s_ggg = 0;
242.     for i = start_idx:end_idx

```

```

243.         t = t_f_ggg(i);
244.         missile_pos = m_t_ggg(i, :);
245.         c_c_ggg = s_p1_ggg + [0, 0, -
v_down * (t - t_e1_ggg)];
246.         if i_f_ggg(missile_pos, a_p_ggg, c_c_ggg, r_s)
247.             o_s_ggg = o_s_ggg + 1;
248.         end
249.     end
250.     o_t_ggg = o_s_ggg * dt;
251. end
252.
253. function i_f_ggg = i_f_ggg(missile_pos, t_p_ggg, s_c_ggg,
r)
254.     n_t_ggg = size(t_p_ggg, 2);
255.     for i = 1:n_t_ggg
256.         target_point = t_p_ggg(:, i)';
257.         i_o_ggg = false;
258.         for j = 1:size(s_c_ggg, 1)
259.             if c_s_ggg(missile_pos, target_point, s_c_ggg(
j, :), r)
260.                 i_o_ggg = true;
261.                 break;
262.             end
263.         end
264.         if ~i_o_ggg
265.             i_f_ggg = false;
266.             return;
267.         end
268.     end
269.     i_f_ggg = true;
270. end
271.
272. function d_i_ggg = c_s_ggg(P, Q, C, r)
273.     v_r_ggg = Q - P; v_p_ggg = C - P;
274.     t = dot(v_p_ggg, v_r_ggg) / dot(v_r_ggg, v_r_ggg);
275.     if t < 0 || t > 1
276.         d_p_ggg = sum(v_p_ggg.^2); d_q_ggg = sum((C - Q).^
2);
277.         d_i_ggg = (d_p_ggg <= r^2) || (d_q_ggg <= r^2);
278.     else
279.         d_s_ggg = sum(v_p_ggg.^2) - (t^2) * dot(v_r_ggg, v
_r_ggg);

```

```

280.         d_i_ggg = (d_s_ggg <= r^2);
281.     end
282. end
283.
284. function p_m_ggg = g_m_ggg(t_v_ggg, M0, v_m, 0)
285.     d_v_ggg = (0 - M0) / norm(0 - M0);
286.     p_m_ggg = M0 + t_v_ggg' * (v_m * d_v_ggg);
287. end
288.
289. function s_i_ggg = s_t_ggg(fitness, t_s_ggg)
290.     p_s_ggg = length(fitness);
291.     indices = randi(p_s_ggg, 1, t_s_ggg);
292.     [~, b_i_ggg] = max(fitness(indices));
293.     s_i_ggg = indices(b_i_ggg);
294. end
295.
296. function individual = m_m_ggg(individual, m_r_ggg, bounds)
297.     n_e_ggg = length(individual);
298.     for i = 1:n_e_ggg
299.         if rand < m_r_ggg
300.             range = bounds.upper(i) - bounds.lower(i);
301.             m_v_ggg = (range * 0.1) * randn;
302.             individual(i) = individual(i) + m_v_ggg;
303.             individual(i) = max(individual(i), bounds.lower
r(i));
304.             individual(i) = min(individual(i), bounds.uppe
r(i));
305.         end
306.     end
307. end

```

附录 7

问题三求解结果

最大总遮蔽时长: 6.74 s

--- 烟雾弹 1 ---

投放时间: 0.00 s

投放后无人机速度: 116.98 m/s

投放后无人机方向 (偏航角): 0.18 rad (10.31 度)

延迟起爆时间: 0.77 s

投放点: (17800.00, 0.00, 1800.00)
起爆点: (17888.19, 16.04, 1797.12)
** 该弹单独有效遮蔽时长: 4.09 s **

--- 烟雾弹 2 ---
投放时间: 2.00 s
投放后无人机速度: 131.17 m/s
投放后无人机方向 (偏航角): 3.30 rad (188.84 度)
延迟起爆时间: 2.02 s
投放点: (18030.17, 41.87, 1800.00)
起爆点: (17768.55, 1.17, 1780.04)
** 该弹单独有效遮蔽时长: 3.29 s **

--- 烟雾弹 3 ---
投放时间: 18.93 s
投放后无人机速度: 95.22 m/s
投放后无人机方向 (偏航角): 0.00 rad (0.00 度)
延迟起爆时间: 2.74 s
投放点: (15835.58, -299.58, 1800.00)
起爆点: (16096.23, -299.58, 1763.28)
** 该弹单独有效遮蔽时长: 0.00 s **

历时 5840.696119 秒。

附录 8

问题四求解 (MATLAB 源码)

```
1. clear; clc;
2. tic; %计时
3.
4. global M0 v_m O T_c r_t h_t v_down r_s t_e_ggg dt g;
5. global A0_1 A0_2 A0_3; %无人机 初始 位置
6.
7. % 导弹 信息
8. M0 = [20000, 0, 2000]; %导弹初始坐标
9. v_m = 300; %导弹速度
10. O = [0, 0, 0];
11.
12. A0_1 = [17800, 0, 1800];
13. A0_2 = [12000, 1400, 1400];
14. A0_3 = [6000, -3000, 700];
15.
16. %真实目标
```

```

17. T_c = [0, 200, 0];
18. r_t = 7; %半径
19. h_t = 10; %高度
20.
21.
22. v_down = 3; %下沉速度
23. r_s = 10; %遮蔽半径
24. t_e_ggg = 20; %遮蔽时长
25. g = 9.8;
26.
27. %时间参数
28. t_start = 0;
29. t_end = 70; %总时长
30. dt = 0.01; %时间 步长
31. t_f_ggg = t_start:dt:t_end;
32.
33. fprintf('进行预计算以加速优化过程...\n');
34. m_t_ggg = g_m_ggg(t_f_ggg, M0, v_m, 0);
35. n_s_ggg = 50;
36. t_v_ggg = linspace(0, 2*pi, n_s_ggg);
37. t_b_ggg = [T_c(1) + r_t * cos(t_v_ggg); T_c(2) + r_t * sin
(t_v_ggg); repmat(T_c(3) - h_t/2, 1, n_s_ggg)];
38. t_t_ggg = [T_c(1) + r_t * sin(t_v_ggg); T_c(2) + r_t * cos
(t_v_ggg); repmat(T_c(3) + h_t/2, 1, n_s_ggg)];
39. a_p_ggg = [t_b_ggg, t_t_ggg];
40.
41.
42. fprintf('开始进行三无人机协同干扰方案优化 (遗传算法)...\n');
43.
44. p_s_ggg = 900; %种群 大小
45. n_g_ggg = 350; %迭代 代数
46. n_e_ggg = 12; %基因 数量
47. c_r_ggg = 0.8; %交叉 概率
48. m_r_ggg = 0.25; %变异 概率
49. e_c_ggg = 2; %精英 个体 数量
50. t_s_ggg = 3; %锦标赛 选择 规模
51.
52.
53. bounds.lower = [70, 0, 0, 0, 70, 0, 0, 0, 70, 0, 0, 0];
54. bounds.upper = [140, 15, 2*pi, 8, 140, 15, 2*pi, 8, 140, 1
5, 2*pi, 8];

```

```

55.
56. p_p_ggg = zeros(p_s_ggg, n_e_ggg);
57. for i = 1:n_e_ggg
58.     p_p_ggg(:, i) = bounds.lower(i) + (bounds.upper(i) - b
ounds.lower(i)) * rand(p_s_ggg, 1);
59. end
60.
61. b_h_ggg = zeros(n_g_ggg, 1);
62. g_f_ggg = -1;
63. g_i_ggg = zeros(1, n_e_ggg);
64.
65. for gen = 1:n_g_ggg
66.
67.     fitness = zeros(p_s_ggg, 1);
68.     for i = 1:p_s_ggg
69.         ind = p_p_ggg(i, :);
70.
71.         params.p1 = struct('v_fy', ind(1), 't_deploy', ind
(2), 'theta_yaw', ind(3), 't_delay', ind(4), 'A0', A0_1);
72.         params.p2 = struct('v_fy', ind(5), 't_deploy', ind
(6), 'theta_yaw', ind(7), 't_delay', ind(8), 'A0', A0_2);
73.         params.p3 = struct('v_fy', ind(9), 't_deploy', ind
(10), 'theta_yaw', ind(11), 't_delay', ind(12), 'A0', A0_3);
74.
75.         fitness(i) = c_o_ggg(params, m_t_ggg, a_p_ggg, t_f
_ggg);
76.     end
77.
78.     [m_c_ggg, idx] = max(fitness);
79.     if m_c_ggg > g_f_ggg
80.         g_f_ggg = m_c_ggg;
81.         g_i_ggg = p_p_ggg(idx, :);
82.     end
83.     b_h_ggg(gen) = g_f_ggg;
84.     fprintf('第 %d 代: 当前最优遮蔽时间 = %.4f s, 全局最
优 = %.4f s\n', gen, m_c_ggg, g_f_ggg);
85.
86.     n_p_ggg = zeros(size(p_p_ggg));
87.     [~, s_i_ggg] = sort(fitness, 'descend');
88.     n_p_ggg(1:e_c_ggg, :) = p_p_ggg(s_i_ggg(1:e_c_ggg), :)
;
89.

```

```

90.     for i = (e_c_ggg + 1):2:p_s_ggg
91.
92.         parent1 = p_p_ggg(s_t_ggg(fitness, t_s_ggg), :);
93.         parent2 = p_p_ggg(s_t_ggg(fitness, t_s_ggg), :);
94.
95.         child1 = parent1; child2 = parent2;
96.         if rand < c_r_ggg
97.             alpha = rand;
98.             child1 = alpha * parent1 + (1 - alpha) * paren
t2;
99.             child2 = alpha * parent2 + (1 - alpha) * paren
t1;
100.        end
101.
102.        n_p_ggg(i, :) = m_m_ggg(child1, m_r_ggg, bounds);
103.        if i+1 <= p_s_ggg
104.            n_p_ggg(i+1, :) = m_m_ggg(child2, m_r_ggg, boun
ds);
105.        end
106.    end
107.    p_p_ggg = n_p_ggg;
108. end
109.
110. fprintf('\n 遗传算法优化完成! \n');
111.
112. best_ind = g_i_ggg;
113. b_p_ggg.p1 = struct('v_fy', best_ind(1), 't_deploy', best_
ind(2), 'theta_yaw', best_ind(3), 't_delay', best_ind(4), 'A0', A0_
1);
114. b_p_ggg.p2 = struct('v_fy', best_ind(5), 't_deploy', best_
ind(6), 'theta_yaw', best_ind(7), 't_delay', best_ind(8), 'A0', A0_
2);
115. b_p_ggg.p3 = struct('v_fy', best_ind(9), 't_deploy', best_
ind(10), 'theta_yaw', best_ind(11), 't_delay', best_ind(12), 'A0',
A0_3);
116.
117. if g_f_ggg > 0
118.     fprintf('最大总遮蔽时长: %.2f s\n\n', g_f_ggg);
119.
120.     uavs = {'FY1', 'FY2', 'FY3'};
121.     ps = {b_p_ggg.p1, b_p_ggg.p2, b_p_ggg.p3};
122.
123.     for i = 1:3

```

```

124.         p = ps{i};
125.         d_v_ggg = [p.v_fy * cos(p.theta_yaw), p.v_fy * sin
(p.theta_yaw), 0];
126.         d_p_ggg = p.A0 + d_v_ggg * p.t_deploy;
127.         e_p_ggg = d_p_ggg + d_v_ggg * p.t_delay + [0, 0, -
0.5 * g * p.t_delay^2];
128.
129.         i_t_ggg = c_i_ggg(p, m_t_ggg, a_p_ggg, t_f_ggg);
130.
131.         fprintf('--- %s 无人机最优参数 ---\n', uavs{i});
132.         fprintf('投放后飞行速度: %.2f m/s\n', p.v_fy);
133.         fprintf('飞行及投放时间: %.2f s\n', p.t_deploy);
134.         fprintf('飞行方向 (偏航
角): %.2f rad (%.2f 度)\n', p.theta_yaw, rad2deg(p.theta_yaw));
135.         fprintf('延迟引爆时间: %.2f s\n', p.t_delay);
136.         fprintf('投放点: (%.2f, %.2f, %.2f)\n', d_p_ggg);
137.         fprintf('起爆点: (%.2f, %.2f, %.2f)\n', e_p_ggg);
138.         fprintf('** 该弹单独有效遮蔽时
长: %.2f s **\n\n', i_t_ggg);
139.     end
140. else
141.         fprintf('未找到有效的遮蔽方案.\n');
142. end
143. toc;
144.
145. figure;
146. plot(1:n_g_ggg, b_h_ggg, 'b-', 'LineWidth', 2);
147. title('遗传算法进化过程 (三无人机协同)');
148. xlabel('代数 (Generation)');
149. ylabel('最优适应度 (最大遮蔽时长 s)');
150. grid on;
151.
152. filename = 'a4.png';
153. saveas(gcf, filename);
154.
155.
156.
157. function o_t_ggg = c_o_ggg(params, m_t_ggg, a_p_ggg, t_f_g
gg)
158.     global v_down r_s t_e_ggg dt g;
159.     p = {params.p1, params.p2, params.p3};
160.     t_exp = zeros(1, 3);

```

```

161.     s_p_ggg = zeros(3, 3);
162.     for i = 1:3
163.         d_v_ggg = [p{i}.v_fy * cos(p{i}.theta_yaw), p{i}.v
_fy * sin(p{i}.theta_yaw), 0];
164.         d_p_ggg = p{i}.A0 + d_v_ggg * p{i}.t_deploy;
165.         t_exp(i) = p{i}.t_deploy + p{i}.t_delay;
166.         s_p_ggg(i, :) = d_p_ggg + d_v_ggg*p{i}.t_delay + [
0, 0, -0.5*g*p{i}.t_delay^2];
167.     end
168.     m_s_ggg = min(t_exp);
169.     m_e_ggg = max(t_exp) + t_e_ggg;
170.     start_idx = floor(m_s_ggg / dt) + 1;
171.     end_idx = min(floor(m_e_ggg / dt) + 1, length(t_f_ggg)
);
172.     if start_idx > end_idx || start_idx < 1
173.         o_t_ggg = 0;
174.         return;
175.     end
176.     o_s_ggg = 0;
177.     for i = start_idx:end_idx
178.         t = t_f_ggg(i);
179.         missile_pos = m_t_ggg(i, :);
180.         a_c_ggg = [];
181.         for j = 1:3
182.             if t >= t_exp(j) && t <= (t_exp(j) + t_e_ggg)
183.                 delta_t = t - t_exp(j);
184.                 c_p_ggg = s_p_ggg(j, :) + [0, 0, -
v_down * delta_t];
185.                 a_c_ggg = [a_c_ggg; c_p_ggg];
186.             end
187.         end
188.         if isempty(a_c_ggg); continue; end
189.         if i_f_ggg(missile_pos, a_p_ggg, a_c_ggg, r_s)
190.             o_s_ggg = o_s_ggg + 1;
191.         end
192.     end
193.     o_t_ggg = o_s_ggg * dt;
194. end
195.
196. function o_t_ggg = c_i_ggg(g_p_ggg, m_t_ggg, a_p_ggg, t_f_
ggg)
197.     global v_down r_s t_e_ggg dt g;

```

```

198.     p1 = g_p_ggg;
199.     d_v_ggg = [p1.v_fy * cos(p1.theta_yaw), p1.v_fy * sin(
p1.theta_yaw), 0];
200.     d_p_ggg = p1.A0 + d_v_ggg * p1.t_deploy;
201.     t_exp1 = p1.t_deploy + p1.t_delay;
202.     s_p_ggg = d_p_ggg + d_v_ggg*p1.t_delay + [0, 0, -
0.5*g*p1.t_delay^2];
203.     t_s_ggg_local = t_exp1;
204.     t_e_ggg_local = t_exp1 + t_e_ggg;
205.     start_idx = floor(t_s_ggg_local / dt) + 1;
206.     end_idx = min(floor(t_e_ggg_local / dt) + 1, length(t_
f_ggg));
207.     if start_idx > end_idx || start_idx < 1; o_t_ggg = 0;
return; end
208.     o_s_ggg = 0;
209.     for i = start_idx:end_idx
210.         t = t_f_ggg(i);
211.         missile_pos = m_t_ggg(i, :);
212.         c_c_ggg = s_p_ggg + [0, 0, -
v_down * (t - t_exp1)];
213.         if i_f_ggg(missile_pos, a_p_ggg, c_c_ggg, r_s)
214.             o_s_ggg = o_s_ggg + 1;
215.         end
216.     end
217.     o_t_ggg = o_s_ggg * dt;
218. end
219.
220. function i_f_ggg = i_f_ggg(missile_pos, t_p_ggg, s_c_ggg,
r)
221.     n_t_ggg = size(t_p_ggg, 2);
222.     n_h_ggg = size(s_c_ggg, 1);
223.     for i = 1:n_t_ggg
224.         target_point = t_p_ggg(:, i)';
225.         i_o_ggg = false;
226.         for j = 1:n_h_ggg
227.             smoke_center = s_c_ggg(j, :);
228.             if c_s_ggg(missile_pos, target_point, smoke_ce
nter, r)
229.                 i_o_ggg = true;
230.                 break;
231.             end
232.         end
233.         if ~i_o_ggg

```

```

234.         i_f_ggg = false;
235.         return;
236.     end
237. end
238.     i_f_ggg = true;
239. end
240.
241. function d_i_ggg = c_s_ggg(P, Q, C, r)
242.     v_r_ggg = Q - P;
243.     v_p_ggg = C - P;
244.     t = dot(v_p_ggg, v_r_ggg) / dot(v_r_ggg, v_r_ggg);
245.     if t < 0 || t > 1
246.         d_p_ggg = sum(v_p_ggg.^2);
247.         d_q_ggg = sum((C - Q).^2);
248.         d_i_ggg = (d_p_ggg <= r^2) || (d_q_ggg <= r^2);
249.     else
250.         d_s_ggg = sum(v_p_ggg.^2) - (t^2) * dot(v_r_ggg, v
_r_ggg);
251.         d_i_ggg = (d_s_ggg <= r^2);
252.     end
253. end
254.
255. function p_m_ggg = g_m_ggg(t_v_ggg, M0, v_m, 0)
256.     d_v_ggg = (0 - M0) / norm(0 - M0);
257.     p_m_ggg = M0 + t_v_ggg' * (v_m * d_v_ggg);
258. end
259.
260.
261. function s_i_ggg = s_t_ggg(fitness, t_s_ggg)
262.     p_s_ggg = length(fitness);
263.     indices = randi(p_s_ggg, 1, t_s_ggg);
264.     [~, b_i_ggg] = max(fitness(indices));
265.     s_i_ggg = indices(b_i_ggg);
266. end
267.
268. function i_i_ggg = m_m_ggg(individual, m_r_ggg, bounds)
269.     n_e_ggg = length(individual);
270.     i_i_ggg = individual;
271.     for i = 1:n_e_ggg
272.         if rand < m_r_ggg
273.             range = bounds.upper(i) - bounds.lower(i);

```


274.	m_v_ggg = (range * 0.1) * randn;
275.	i_i_ggg(i) = i_i_ggg(i) + m_v_ggg;
276.	i_i_ggg(i) = max(i_i_ggg(i), bounds.lower(i));
277.	i_i_ggg(i) = min(i_i_ggg(i), bounds.upper(i));
278.	end
279.	end
280.	end

附录 9
问题四求解结果
遗传算法优化完成！ 最大总遮蔽时长: 8.32 s --- FY1 无人机最优参数 --- 投放后飞行速度: 129.46 m/s 飞行及投放时间: 0.04 s 飞行方向 (偏航角): 0.11 rad (6.47 度) 延迟引爆时间: 0.66 s 投放点: (17805.68, 0.64, 1800.00) 起爆点: (17890.18, 10.23, 1797.89) ** 该弹单独有效遮蔽时长: 4.52 s ** --- FY2 无人机最优参数 --- 投放后飞行速度: 88.38 m/s 飞行及投放时间: 10.23 s 飞行方向 (偏航角): 4.95 rad (283.81 度) 延迟引爆时间: 5.71 s 投放点: (12215.78, 522.42, 1400.00) 起爆点: (12336.35, 32.06, 1240.03) ** 该弹单独有效遮蔽时长: 3.80 s ** --- FY3 无人机最优参数 --- 投放后飞行速度: 121.03 m/s 飞行及投放时间: 1.07 s 飞行方向 (偏航角): 4.29 rad (245.82 度) 延迟引爆时间: 0.82 s 投放点: (5946.82, -3118.43, 700.00) 起爆点: (5906.18, -3208.94, 696.71) ** 该弹单独有效遮蔽时长: 0.00 s ** 历时 6386.422598 秒。

附录 10

问题五求解 (PYTHON 源码)

```
1.  import numpy as np
2.  import numba
3.  from numba import jit, prange
4.  import time
5.  import matplotlib.pyplot as plt
6.  from scipy.spatial.distance import cdist
7.
8.  M_P_GGG = np.array([
9.      [20000.0, 0.0, 2000.0],
10.     [19000.0, 600.0, 2100.0],
11.     [18000.0, -600.0, 1900.0]
12. ])
13.
14. U_P_GGG = np.array([
15.     [17800.0, 0.0, 1800.0],
16.     [12000.0, 1400.0, 1400.0],
17.     [6000.0, -3000.0, 700.0],
18.     [11000.0, 2000.0, 1800.0],
19.     [13000.0, -2000.0, 1300.0]
20. ])
21.
22. T_C_GGG = np.array([0.0, 200.0, 0.0])
23. T_R_GGG = 7.0
24. T_H_GGG = 10.0
25.
26. M_S_GGG = 300.0
27. U_N_GGG = 70.0
28. U_X_GGG = 140.0
29. S_K_GGG = 3.0
30. S_R_GGG = 10.0
31. S_T_GGG = 20.0
32. G_G_GGG = 9.8
33.
34. S_A_GGG = 0.0
35. S_E_GGG = 100.0
36. T_E_GGG = 0.01
37.
38. P_S_GGG = 800
```

```

39. N_G_GGG = 300
40. C_R_GGG = 0.85
41. M_R_GGG = 0.25
42. E_C_GGG = 2
43.
44. N_S_GGG = 100
45.
46. def g_t_ggg():
47.     theta = np.linspace(0, 2 * np.pi, N_S_GGG)
48.     b_c_ggg = np.column_stack([
49.         T_C_GGG[0] + T_R_GGG * np.cos(theta),
50.         T_C_GGG[1] + T_R_GGG * np.sin(theta),
51.         np.full(N_S_GGG, T_C_GGG[2] - T_H_GGG / 2)
52.     ])
53.     t_c_ggg = np.column_stack([
54.         T_C_GGG[0] + T_R_GGG * np.cos(theta),
55.         T_C_GGG[1] + T_R_GGG * np.sin(theta),
56.         np.full(N_S_GGG, T_C_GGG[2] + T_H_GGG / 2)
57.     ])
58.     return np.vstack([b_c_ggg, t_c_ggg])
59.
60. T_A_GGG = g_t_ggg()
61.
62. @jit(nopython=True)
63. def c_m_ggg():
64.     t_s_ggg = int((S_E_GGG - S_A_GGG) / T_E_GGG) + 1
65.     t_r_ggg = np.zeros((3, t_s_ggg, 3))
66.     for i in range(3):
67.         i_p_ggg = M_P_GGG[i]
68.         n_v_ggg = np.sqrt(i_p_ggg[0] ** 2 + i_p_ggg[1] **
2 + i_p_ggg[2] ** 2)
69.         direction = -i_p_ggg / n_v_ggg
70.         for t_idx in range(t_s_ggg):
71.             t = S_A_GGG + t_idx * T_E_GGG
72.             t_r_ggg[i, t_idx] = i_p_ggg + direction * M_S_
GGG * t
73.     return t_r_ggg
74.
75. M_T_GGG = c_m_ggg()
76.
77. @jit(nopython=True)
78. def p_d_ggg(point, l_s_ggg, l_e_ggg):

```

```

79.     l_v_ggg = l_e_ggg - l_s_ggg
80.     p_v_ggg = point - l_s_ggg
81.     l_l_ggg = np.sqrt(l_v_ggg[0] ** 2 + l_v_ggg[1] ** 2 +
l_v_ggg[2] ** 2)
82.     l_u_ggg = l_v_ggg / l_l_ggg
83.     p_s_ggg = p_v_ggg / l_l_ggg
84.     t = l_u_ggg[0] * p_s_ggg[0] + l_u_ggg[1] * p_s_ggg[1]
+ l_u_ggg[2] * p_s_ggg[2]
85.     t = max(0.0, min(1.0, t))
86.     n_p_ggg = l_s_ggg + t * l_v_ggg
87.     dx = n_p_ggg[0] - point[0]
88.     dy = n_p_ggg[1] - point[1]
89.     dz = n_p_ggg[2] - point[2]
90.     return np.sqrt(dx * dx + dy * dy + dz * dz)
91.
92. @jit(nopython=True)
93. def i_o_ggg(m_p_ggg, t_p_ggg, s_c_ggg, s_r_ggg):
94.     return p_d_ggg(s_c_ggg, m_p_ggg, t_p_ggg) <= s_r_ggg
95.
96. @jit(nopython=True)
97. def i_t_ggg(m_p_ggg, s_c_ggg, s_r_ggg):
98.     for i in range(T_A_GGG.shape[0]):
99.         t_p_ggg = T_A_GGG[i]
100.        obscured = False
101.        for j in range(s_c_ggg.shape[0]):
102.            if i_o_ggg(m_p_ggg, t_p_ggg, s_c_ggg[j], s_r_g
gg):
103.                obscured = True
104.                break
105.            if not obscured:
106.                return False
107.        return True
108.
109. @jit(nopython=True)
110. def c_s_ggg(u_p_ggg, uav_idx, smoke_idx):
111.     p_o_ggg = (uav_idx * 3 + smoke_idx) * 4
112.     speed = u_p_ggg[p_o_ggg]
113.     d_d_ggg = u_p_ggg[p_o_ggg + 1]
114.     d_t_ggg = u_p_ggg[p_o_ggg + 2]
115.     d_e_ggg = u_p_ggg[p_o_ggg + 3]
116.     d_r_ggg = np.deg2rad(d_d_ggg)
117.     u_i_ggg = U_P_GGG[uav_idx]

```

```

118.     u_d_ggg = np.array([np.cos(d_r_ggg), np.sin(d_r_ggg),
0.0])
119.     d_p_ggg = u_i_ggg + u_d_ggg * speed * d_t_ggg
120.     e_t_ggg = d_t_ggg + d_e_ggg
121.     e_p_ggg = d_p_ggg + u_d_ggg * speed * d_e_ggg
122.     e_p_ggg[2] -= 0.5 * G_G_GGG * d_e_ggg ** 2
123.     e_s_ggg = e_t_ggg
124.     e_e_ggg = e_t_ggg + S_T_GGG
125.     return d_p_ggg, e_p_ggg, e_s_ggg, e_e_ggg
126.
127. @jit(nopython=True)
128. def c_i_ggg(individual):
129.     t_i_ggg = 0.0
130.     num_uavs = 5
131.     n_s_ggg = 3
132.     t_s_ggg = num_uavs * n_s_ggg
133.     s_p_ggg = np.zeros((t_s_ggg, 5))
134.     for uav_idx in range(num_uavs):
135.         for smoke_idx in range(n_s_ggg):
136.             d_p_ggg, e_p_ggg, e_s_ggg, e_e_ggg = c_s_ggg(
137.                 individual, uav_idx, smoke_idx
138.             )
139.             param_idx = uav_idx * n_s_ggg + smoke_idx
140.             s_p_ggg[param_idx, 0:3] = e_p_ggg
141.             s_p_ggg[param_idx, 3] = e_s_ggg
142.             s_p_ggg[param_idx, 4] = e_e_ggg
143.         a_s_ggg = np.zeros((t_s_ggg, 3))
144.         for t_idx in range(M_T_GGG.shape[1]):
145.             t = S_A_GGG + t_idx * T_E_GGG
146.             n_a_ggg = 0
147.             for i in range(t_s_ggg):
148.                 e_p_ggg = s_p_ggg[i, 0:3]
149.                 e_s_ggg = s_p_ggg[i, 3]
150.                 e_e_ggg = s_p_ggg[i, 4]
151.                 if e_s_ggg <= t <= e_e_ggg:
152.                     s_d_ggg = S_K_GGG * (t - e_s_ggg)
153.                     s_c_ggg = e_p_ggg.copy()
154.                     s_c_ggg[2] -= s_d_ggg
155.                     a_s_ggg[n_a_ggg] = s_c_ggg
156.                     n_a_ggg += 1
157.             if n_a_ggg == 0:

```

```

158.         continue
159.         v_s_ggg = a_s_ggg[:n_a_ggg]
160.         for missile_idx in range(3):
161.             m_p_ggg = M_T_GGG[missile_idx, t_idx]
162.             if i_t_ggg(m_p_ggg, v_s_ggg, S_R_GGG):
163.                 t_i_ggg += T_E_GGG
164.         return t_i_ggg
165.
166. def i_p_ggg():
167.     population = np.zeros((P_S_GGG, 60))
168.     for i in range(P_S_GGG):
169.         for j in range(60):
170.             if j % 4 == 0:
171.                 population[i, j] = np.random.uniform(U_N_G
GG, U_X_GGG)
172.             elif j % 4 == 1:
173.                 population[i, j] = np.random.uniform(0, 36
0)
174.             elif j % 4 == 2:
175.                 population[i, j] = np.random.uniform(0, 10
)
176.             else:
177.                 population[i, j] = np.random.uniform(0, 8)
178.         return population
179.
180. def crossover(parent1, parent2):
181.     if np.random.rand() < C_R_GGG:
182.         c_p_ggg = np.random.randint(1, len(parent1) - 1)
183.         child1 = np.concatenate([parent1[:c_p_ggg], parent
2[c_p_ggg:]])
184.         child2 = np.concatenate([parent2[:c_p_ggg], parent
1[c_p_ggg:]])
185.         return child1, child2
186.     return parent1.copy(), parent2.copy()
187.
188. def mutate(individual):
189.     for i in range(len(individual)):
190.         if np.random.rand() < M_R_GGG:
191.             if i % 4 == 0:
192.                 individual[i] = np.clip(individual[i] + np
.random.normal(0, 5), U_N_GGG, U_X_GGG)
193.             elif i % 4 == 1:

```

```

194.             individual[i] = np.clip(individual[i] + np
.random.normal(0, 10), 0, 360)
195.             elif i % 4 == 2:
196.                 individual[i] = np.clip(individual[i] + np
.random.normal(0, 1), 0, 10)
197.             else:
198.                 individual[i] = np.clip(individual[i] + np
.random.normal(0, 0.5), 0, 8)
199.         return individual
200.
201. @jit(nopython=True, parallel=True)
202. def c_f_ggg(population):
203.     fitness = np.zeros(population.shape[0])
204.     for i in prange(population.shape[0]):
205.         fitness[i] = c_i_ggg(population[i])
206.     return fitness
207.
208. def g_a_ggg():
209.     population = i_p_ggg()
210.     b_h_ggg = []
211.     b_i_ggg = None
212.     b_f_ggg = -1
213.     for generation in range(N_G_GGG):
214.         s_t_ggg = time.time()
215.         fitness = c_f_ggg(population)
216.         m_f_ggg = np.argmax(fitness)
217.         if fitness[m_f_ggg] > b_f_ggg:
218.             b_f_ggg = fitness[m_f_ggg]
219.             b_i_ggg = population[m_f_ggg].copy()
220.             b_h_ggg.append(b_f_ggg)
221.             e_i_ggg = np.argsort(fitness)[-E_C_GGG:]
222.             n_p_ggg = population[e_i_ggg].copy()
223.             while len(n_p_ggg) < P_S_GGG:
224.                 t_s_ggg = 3
225.                 t_i_ggg = np.random.choice(P_S_GGG, t_s_ggg, r
eplace=False)
226.                 t_f_ggg = fitness[t_i_ggg]
227.                 p1_idx = t_i_ggg[np.argmax(t_f_ggg)]
228.                 t_i_ggg = np.random.choice(P_S_GGG, t_s_ggg, r
eplace=False)
229.                 t_f_ggg = fitness[t_i_ggg]
230.                 p2_idx = t_i_ggg[np.argmax(t_f_ggg)]

```

```

231.         child1, child2 = crossover(population[p1_idx],
population[p2_idx])
232.         child1 = mutate(child1)
233.         child2 = mutate(child2)
234.         n_p_ggg = np.vstack([n_p_ggg, child1])
235.         if len(n_p_ggg) < P_S_GGG:
236.             n_p_ggg = np.vstack([n_p_ggg, child2])
237.         population = n_p_ggg
238.         e_t_ggg = time.time()
239.         print(
240.             f"Generation {generation + 1}/{N_G_GGG}, Best
Fitness: {b_f_ggg:.2f}, Time: {e_t_ggg - s_t_ggg:.2f}s")
241.         return b_i_ggg, b_f_ggg, b_h_ggg
242.
243. def a_s_ggg(b_i_ggg):
244.     print("最优解分析:")
245.     print("=" * 50)
246.     s_d_ggg = []
247.     for uav_idx in range(5):
248.         for smoke_idx in range(3):
249.             p_o_ggg = (uav_idx * 3 + smoke_idx) * 4
250.             speed = b_i_ggg[p_o_ggg]
251.             d_d_ggg = b_i_ggg[p_o_ggg + 1]
252.             d_t_ggg = b_i_ggg[p_o_ggg + 2]
253.             d_e_ggg = b_i_ggg[p_o_ggg + 3]
254.             d_r_ggg = np.deg2rad(d_d_ggg)
255.             u_i_ggg = U_P_GGG[uav_idx]
256.             u_d_ggg = np.array([np.cos(d_r_ggg), np.sin(d_
r_ggg), 0.0])
257.             d_p_ggg = u_i_ggg + u_d_ggg * speed * d_t_ggg
258.             e_p_ggg = d_p_ggg + u_d_ggg * speed * d_e_ggg
259.             e_p_ggg[2] -= 0.5 * G_G_GGG * d_e_ggg ** 2
260.             e_s_ggg = d_t_ggg + d_e_ggg
261.             e_e_ggg = e_s_ggg + S_T_GGG
262.             o_t_ggg = np.zeros(3)
263.             for missile_idx in range(3):
264.                 o_d_ggg = 0
265.                 for t_idx in range(M_T_GGG.shape[1]):
266.                     t = S_A_GGG + t_idx * T_E_GGG
267.                     if e_s_ggg <= t <= e_e_ggg:
268.                         s_k_ggg = S_K_GGG * (t - e_s_ggg)
269.                         s_c_ggg = e_p_ggg.copy()

```



```

270.                s_c_ggg[2] -= s_k_ggg
271.                m_p_ggg = M_T_GGG[missile_idx, t_i
dx]
272.                s_c_array_ggg = np.array([s_c_ggg]
)
273.                if i_t_ggg(m_p_ggg, s_c_array_ggg,
S_R_GGG):
274.                    o_d_ggg += T_E_GGG
275.                    o_t_ggg[missile_idx] = o_d_ggg
276.                    s_d_ggg.append({
277.                        'uav': uav_idx + 1,
278.                        'smoke': smoke_idx + 1,
279.                        'speed': speed,
280.                        'direction': d_d_ggg,
281.                        'deploy_time': d_t_ggg,
282.                        'detonation_delay': d_e_ggg,
283.                        'deploy_point': d_p_ggg,
284.                        'explosion_point': e_p_ggg,
285.                        'obscured_times': o_t_ggg
286.                    })
287.                    print(f"UAV {uav_idx + 1} 烟雾
弹 {smoke_idx + 1}:")
288.                    print(f"    速度: {speed:.2f} m/s, 方
向: {d_d_ggg:.2f}°")
289.                    print(f"    投放时间: {d_t_ggg:.2f} s, 引爆延
迟: {d_e_ggg:.2f} s")
290.                    print(f"    投放
点: ({d_p_ggg[0]:.2f}, {d_p_ggg[1]:.2f}, {d_p_ggg[2]:.2f})")
291.                    print(f"    爆炸
点: ({e_p_ggg[0]:.2f}, {e_p_ggg[1]:.2f}, {e_p_ggg[2]:.2f})")
292.                    print(f"    对 M1 遮蔽时间: {o_t_ggg[0]:.2f} s")
293.                    print(f"    对 M2 遮蔽时间: {o_t_ggg[1]:.2f} s")
294.                    print(f"    对 M3 遮蔽时间: {o_t_ggg[2]:.2f} s")
295.                    print()
296.                    t_o_ggg = c_i_ggg(b_i_ggg)
297.                    print(f"\n 最终适应度得分 (各导弹遮蔽时间总
和): {t_o_ggg:.2f} s")
298.                    return s_d_ggg, t_o_ggg
299.
300. def main():
301.     print("开始优化...")
302.     s_m_ggg = time.time()
303.     b_i_ggg, b_f_ggg, f_h_ggg = g_a_ggg()

```

```

304.     e_m_ggg = time.time()
305.     print(f"优化完成, 耗时: {e_m_ggg - s_m_ggg:.2f} 秒")
306.     print(f"最佳适应度: {b_f_ggg:.2f}")
307.     s_d_ggg, t_o_ggg = a_s_ggg(b_i_ggg)
308.     plt.figure(figsize=(10, 6))
309.     plt.plot(range(1, N_G_GGG + 1), f_h_ggg)
310.     plt.xlabel('Generation')
311.     plt.ylabel('Best Fitness (Obscured Time)')
312.     plt.title('Genetic Algorithm Evolution')
313.     plt.grid(True)
314.     plt.savefig('evolution.png')
315.     plt.show()
316.     return b_i_ggg, s_d_ggg, t_o_ggg
317.
318. if __name__ == "__main__":
319.     b_i_ggg, s_d_ggg, t_o_ggg = main()

```

附录 11

问题五求解结果

最佳适应度: 13.85

最优解分析:

=====

UAV 1 烟雾弹 1:

速度: 97.11 m/s, 方向: 9.26°
 投放时间: 1.09 s, 引爆延迟: 0.00 s
 投放点: (17904.20, 16.98, 1800.00)
 爆炸点: (17904.20, 16.98, 1800.00)
 对 M1 遮蔽时间: 3.61 s
 对 M2 遮蔽时间: 0.00 s
 对 M3 遮蔽时间: 0.00 s

UAV 1 烟雾弹 2:

速度: 80.14 m/s, 方向: 150.64°
 投放时间: 8.16 s, 引爆延迟: 5.68 s
 投放点: (17230.30, 320.43, 1800.00)
 爆炸点: (16833.85, 543.41, 1642.14)
 对 M1 遮蔽时间: 0.00 s
 对 M2 遮蔽时间: 0.00 s
 对 M3 遮蔽时间: 0.00 s

UAV 1 烟雾弹 3:

速度: 105.52 m/s, 方向: 53.67°

投放时间: 0.00 s, 引爆延迟: 0.04 s
投放点: (17800.00, 0.00, 1800.00)
爆炸点: (17802.35, 3.20, 1799.99)
对 M1 遮蔽时间: 3.10 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 2 烟雾弹 1:

速度: 117.46 m/s, 方向: 330.93°
投放时间: 3.04 s, 引爆延迟: 5.07 s
投放点: (12311.92, 1226.62, 1400.00)
爆炸点: (12832.17, 937.44, 1274.16)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 2 烟雾弹 2:

速度: 136.32 m/s, 方向: 286.10°
投放时间: 4.57 s, 引爆延迟: 2.80 s
投放点: (12172.74, 801.37, 1400.00)
爆炸点: (12278.42, 435.11, 1361.69)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 3.82 s
对 M3 遮蔽时间: 0.00 s

UAV 2 烟雾弹 3:

速度: 140.00 m/s, 方向: 293.11°
投放时间: 7.08 s, 引爆延迟: 0.56 s
投放点: (12389.07, 488.45, 1400.00)
爆炸点: (12419.63, 416.83, 1398.48)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 3.76 s
对 M3 遮蔽时间: 0.00 s

UAV 3 烟雾弹 1:

速度: 119.81 m/s, 方向: 13.48°
投放时间: 4.63 s, 引爆延迟: 0.85 s
投放点: (6539.60, -2870.69, 700.00)
爆炸点: (6638.85, -2846.90, 696.44)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 3 烟雾弹 2:

速度: 125.22 m/s, 方向: 171.70°

投放时间: 2.29 s, 引爆延迟: 2.76 s
投放点: (5716.81, -2958.67, 700.00)
爆炸点: (5374.29, -2908.67, 662.56)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 3 烟雾弹 3:

速度: 99.68 m/s, 方向: 284.12°
投放时间: 10.00 s, 引爆延迟: 6.77 s
投放点: (6243.16, -3966.73, 700.00)
爆炸点: (6407.84, -4621.46, 475.25)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 4 烟雾弹 1:

速度: 130.82 m/s, 方向: 332.59°
投放时间: 1.00 s, 引爆延迟: 0.55 s
投放点: (11115.65, 1940.04, 1800.00)
爆炸点: (11179.74, 1906.81, 1798.51)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 4 烟雾弹 2:

速度: 108.89 m/s, 方向: 345.14°
投放时间: 10.00 s, 引爆延迟: 3.92 s
投放点: (12052.49, 1720.73, 1800.00)
爆炸点: (12465.08, 1611.25, 1724.70)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 4 烟雾弹 3:

速度: 72.60 m/s, 方向: 215.07°
投放时间: 6.57 s, 引爆延迟: 4.11 s
投放点: (10609.64, 1725.95, 1800.00)
爆炸点: (10365.46, 1554.53, 1717.24)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 5 烟雾弹 1:

速度: 135.31 m/s, 方向: 360.00°

投放时间: 6.98 s, 引爆延迟: 0.61 s
投放点: (13944.73, -2000.00, 1300.00)
爆炸点: (14027.87, -2000.00, 1298.15)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 5 烟雾弹 2:

速度: 83.93 m/s, 方向: 327.18°
投放时间: 2.16 s, 引爆延迟: 3.23 s
投放点: (13152.09, -2098.07, 1300.00)
爆炸点: (13379.94, -2245.01, 1248.86)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

UAV 5 烟雾弹 3:

速度: 78.28 m/s, 方向: 17.96°
投放时间: 3.34 s, 引爆延迟: 5.57 s
投放点: (13248.53, -1919.43, 1300.00)
爆炸点: (13662.97, -1785.07, 1148.22)
对 M1 遮蔽时间: 0.00 s
对 M2 遮蔽时间: 0.00 s
对 M3 遮蔽时间: 0.00 s

最终适应度得分 (各导弹遮蔽时间总和): 13.85 s